

Vellore Institute of Technology (Chennai Campus)

Options Trading Data Science Project Report

Data Analytics Solutions using R

SUBMITTED BY

Harsh Vardhan Ajmera (24BDS1060)

Ahana Manna (24BDS1102)

SUBMITTED TO

Dr. Vijayalakshmi A

1. Title

Options Trading Data Science Project (Study Stock): from recorded NSE tick data to a 15-model R benchmark.

2. Team Members

Harsh Vardhan Ajmera (24BDS1060) and Ahana Manna (24BDS1102). **Faculty:** Dr. Vijayalakshmi A, VIT Chennai.

3. Abstract

Most people who trade index options in India lose money, and a big reason is that every trade pays the bid-ask spread and a stack of government charges before it can make a rupee. We wanted to see whether a machine learning model could act as a filter that says "take this trade" or "skip it" once those costs are counted.

We recorded live NIFTY and BANKNIFTY option ticks from the NSE through the Upstox WebSocket feed and stored them as per-minute files on our own PC. From that data we built a 500-signal dataset with five inputs: confidence, spread, log volume, a volume-weighted open-interest momentum score and an open-interest flow score. In R we cleaned the data, explored it, ran statistical tests, and compared 15 classifiers on a time-ordered test set of 94 signals. Accuracy ranged from 89.4% (single decision tree) to 100% (probit regression and a linear SVM).

> **An honest caveat.** In this dataset the "profitable" label is calculated from the same five inputs, so the models are mostly rediscovering our formula. The scores show that the pipeline works, not that the market can be predicted. Our drafted research paper tests the harder question with labels taken from real future prices and finds an AUC of at most 0.60 and no profitable filter after costs.

4. Introduction

Index options on the NSE are some of the most heavily traded contracts in the world, and most of that volume comes from individual traders holding positions for a few minutes. At that horizon the price move is usually small compared with what the trade costs: the spread on the way in and out, brokerage, exchange charges, securities transaction tax, stamp duty, the SEBI fee and GST. A trader can be right about direction and still lose.

Study Stock is the platform we built around this problem. A Python engine records market data and computes the Greeks, a Node gateway serves it, and a React web app shows a live options chain, a paper-trading Demo Lab and a research page. This report covers the data-analysis side, done in R.

What we set out to do

- Record real NSE option data and keep it on our own machine.

- Clean it and understand it through plots and statistical tests.
- Compare 15 machine learning models fairly, on data from after the training period.
- Show the working application, and be upfront about what the results do and do not prove.

5. Data Set Description

All data comes from our own collector, which listens to the Upstox WebSocket feed for NIFTY and BANKNIFTY options and writes one Parquet file per minute. For R we exported four small CSV samples into the project's `data/` folder:

- `raw_sample_dataset.csv` – 1,000 raw ticks, 19 columns (price, bid, ask, volume, open interest, exchange time, feed latency). Saved 12 Oct 2026, 181,044 bytes.
- `preprocessed_features_sample.csv` – 1,000 cleaned option-chain rows, 20 columns (spread, mid price, change in open interest, 15-minute z-scores, distance from the money). Saved 12 Oct 2026, 171,732 bytes.
- `r_cleaned_features_output.csv` – 60 one-minute bars produced by our R cleaning script, 25 columns. Saved 13 Oct 2026, 14,320 bytes.
- `options_trade_signals_sample.csv` – the **modelling dataset**: 500 trade signals, 12 columns, no missing values. Saved 12 Oct 2026, 44,424 bytes.

The 500 signals are 374 NIFTY and 126 BANKNIFTY, 255 calls and 245 puts, all for the 30 June 2026 expiry. The five model inputs are `confidence`, `spread`, `log_volume`, `z_vwoim` and `z_oi_flow`.

How the target is defined. Each signal gets a cost: `friction = 0.35 × spread + 0.15`, and an expected gain: `0.08 × z_vwoim + 0.06 × z_oi_flow + 0.20 × (confidence − 0.90)`. If gain minus cost is above zero the signal is labelled Profitable (255 signals, 51%), otherwise Unprofitable (245 signals, 49%).

How we split it. Signals are sorted by time. The earliest 80% (400 signals) train the models and the latest part (94 signals) is the test set, with a 2% gap left out in between so that neighbouring ticks cannot leak from one side to the other.

> Because the label is a formula of the inputs, very high accuracy is expected. We say this plainly here and again in the conclusion.

Proof that the data is stored on our PC

The full recording sits in `C:\Harsh\StockProject\HARSH\OPTIONS!!!!!\`. We counted it directly from the folder: **57,740 per-minute Parquet files in 48 session folders**, from 18 June 2026 to 20 October 2026. We measured the size of only the first 13 session folders, about 955 MB. The four CSV sizes and dates above were read from the file system. The research paper analyses 69,519,707 ticks over 36 sessions, which is a different cut of the same recording from the 48 folders counted today.

6. Exploratory Data Analysis and Visualizations

All plots were made in R with ggplot2.

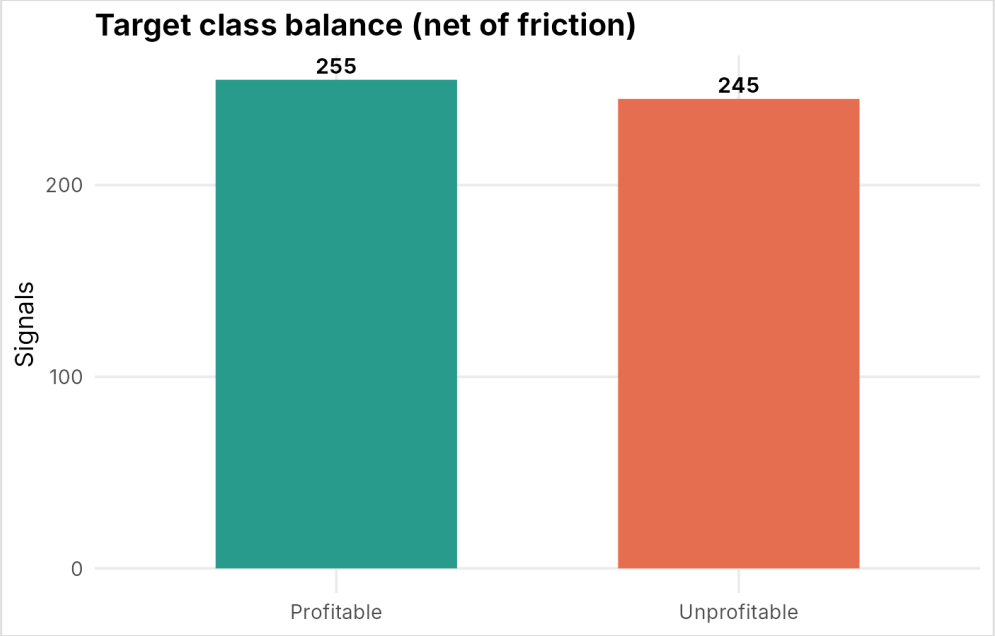


Figure 1. The classes are close to balanced (255 vs 245), so plain accuracy is a fair first measure.

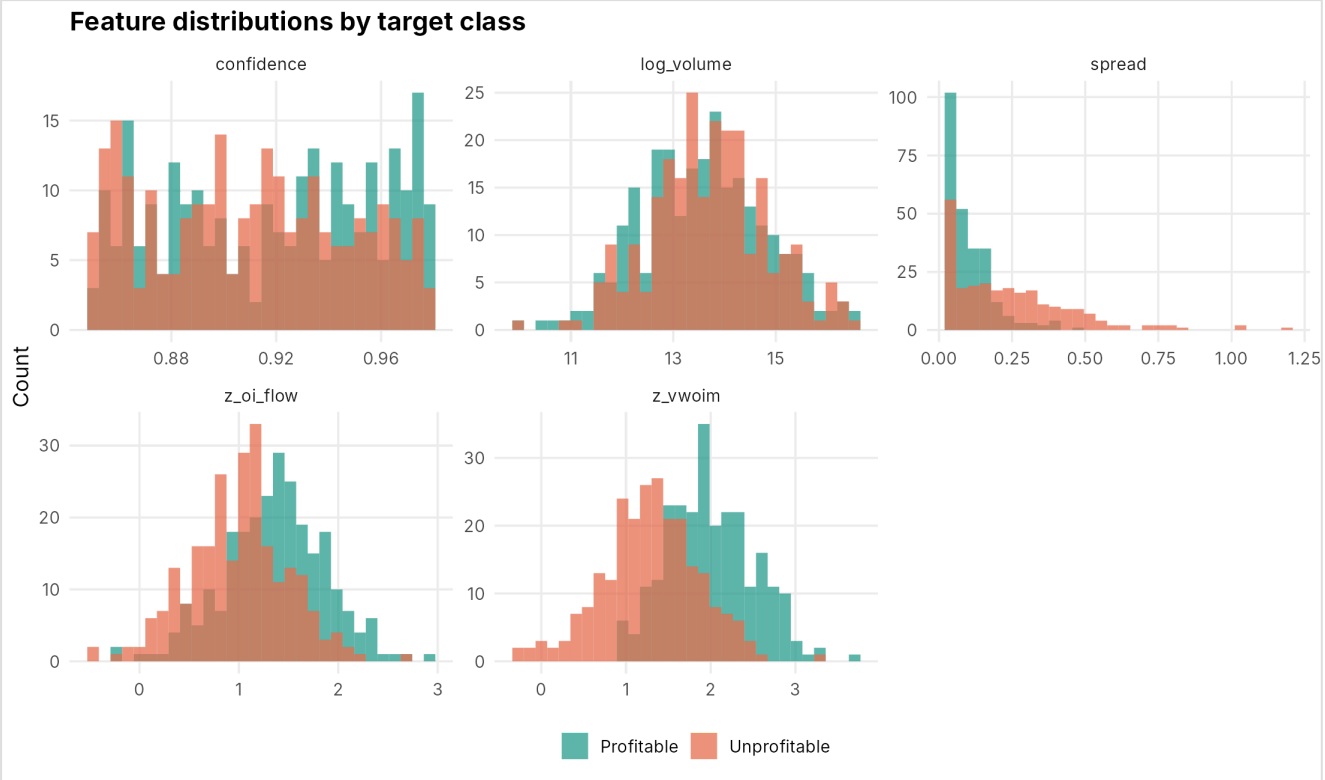


Figure 2. Profitable signals sit at higher z_vwoim and z_oi_flow and lower spread. Spread is strongly right-skewed.

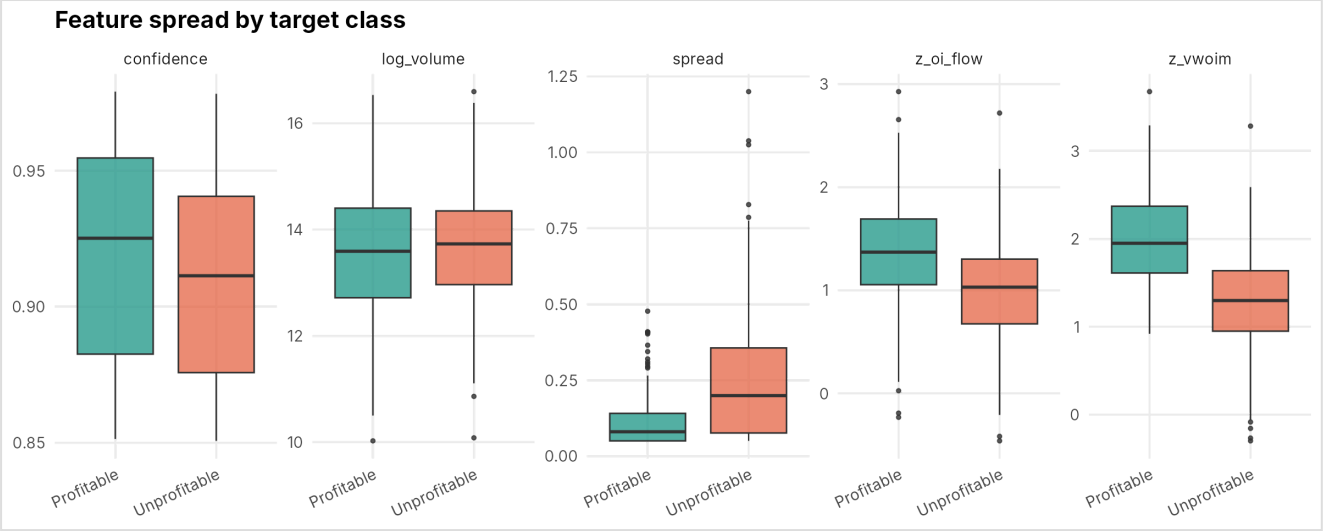


Figure 3. Unprofitable signals have clearly wider spreads; z_vwoim shows the biggest gap between classes; volume barely differs.

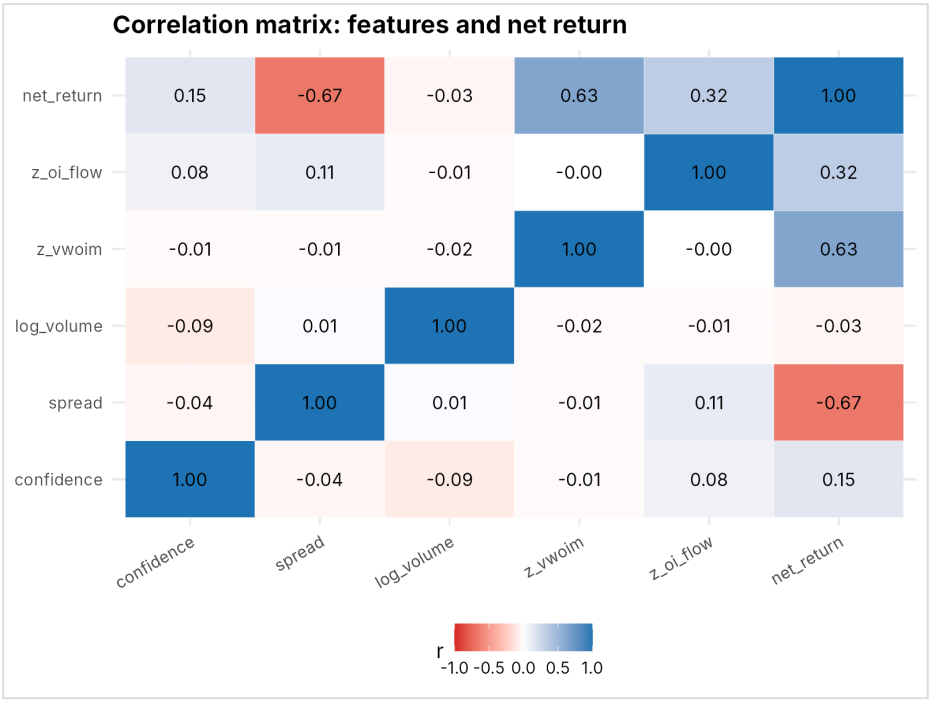


Figure 4. Net return moves with spread (r = -0.67) and z_vwoim (r = 0.63). The inputs are almost uncorrelated with each other.

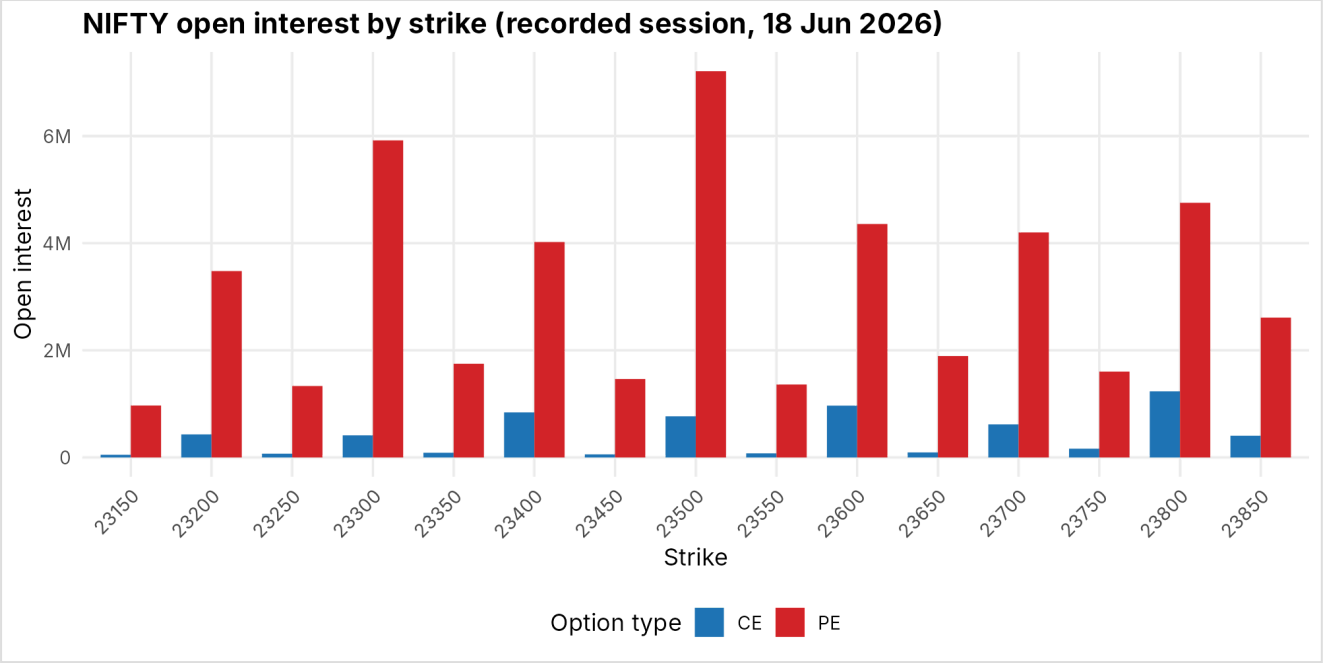


Figure 5. NIFTY open interest by strike from the recorded session of 18 June 2026.

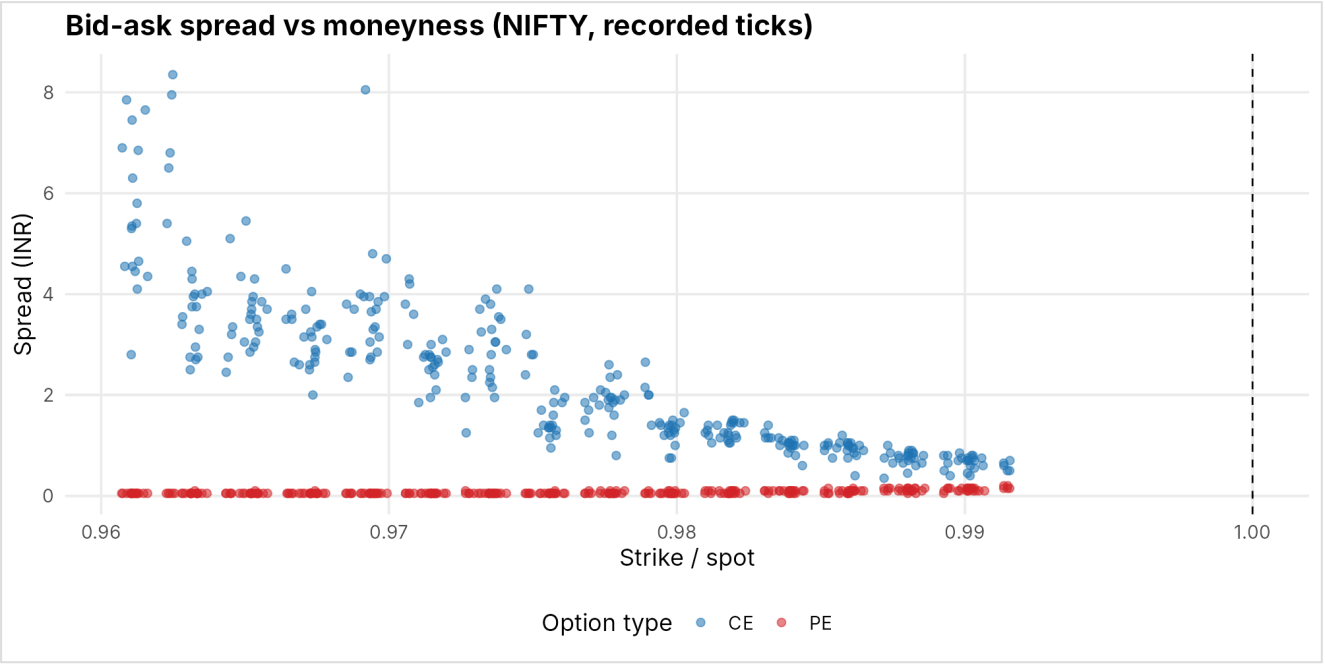


Figure 6. Spreads widen for strikes far from the money, which is why spread matters so much as a cost.

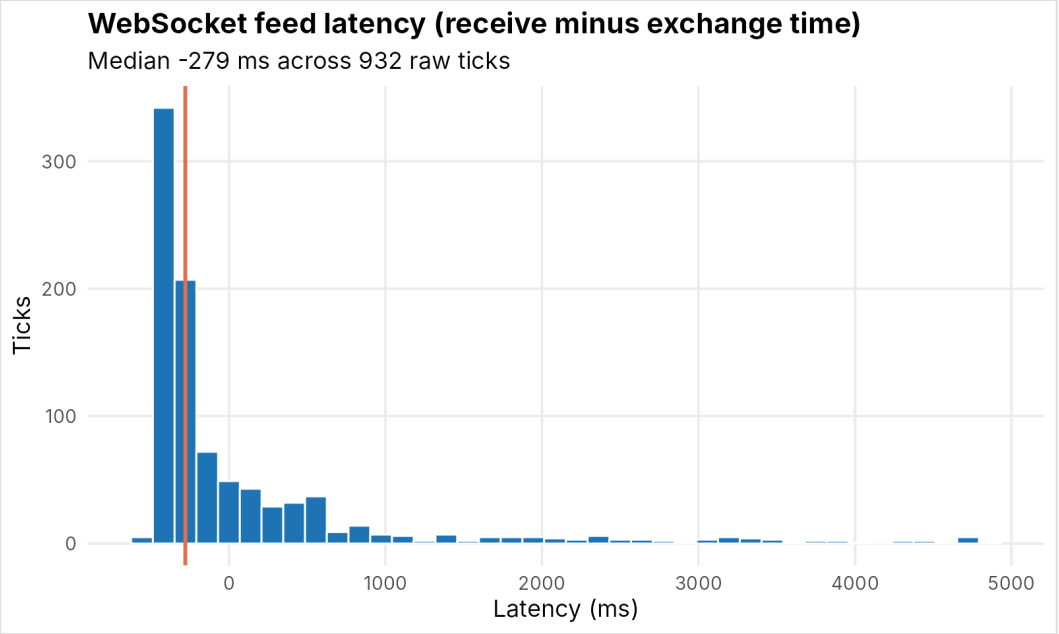


Figure 7. Feed latency (receive time minus exchange time). Some values are negative because our PC clock and the exchange clock are not perfectly in sync, so read this as the shape of the delay, not an exact figure.

7. Statistical Analysis

We used a 5% significance level. The full R output is shown below the summary.

- **Descriptive statistics.** Spread averages Rs 0.18 (median 0.115) and is strongly right-skewed (skewness 2.17). The other four inputs are roughly symmetric.
- **Profitable vs unprofitable (Welch t-test and Wilcoxon test).** Spread, z_vwoim and z_oi_flow differ very strongly between the classes (p below 1e-14 in every case). Confidence differs weakly (p about 0.01). Trading volume does not differ (p about 0.24).
- **Chi-square tests.** Option type (call or put) and underlying (NIFTY or BANKNIFTY) are unrelated to the label (p = 1.00 and p = 0.64).
- **ANOVA.** Average spread is the same for NIFTY and BANKNIFTY (p = 0.885).
- **Shapiro-Wilk.** Spread and confidence are not normally distributed, so for them we trust the Wilcoxon test, which agreed with the t-test. The other three inputs look normal.
- **Correlation with net return.** Spread -0.667, z_vwoim 0.630, z_oi_flow 0.319, confidence 0.145.

These patterns are what we should see when a label is built from the inputs, so we treat them as a check that the pipeline is correct, not as a finding about the market.

R output

```
=== DATASET OVERVIEW ===
Rows: 500  Columns: 12
Underlyings:

BANKNIFTY     NIFTY
      126       374
Option type:

 CE  PE
255 245
Expiries:
```

```
2026-06-30
       500
Missing values per column (modelling dataset):
      timestamp      underlying          expiry          strike     option_type
              0               0               0               0               0
     confidence          spread          volume         z_vwoim       z_oi_flow
              0               0               0               0               0
 times_repeated execution_score
              0               0

Class distribution of target:

  Profitable Unprofitable
         255          245

  Profitable Unprofitable
        0.51         0.49

=== DESCRIPTIVE STATISTICS (feature columns) ===
     feature    mean      sd  median     min     max     skew
1 confidence  0.9148 0.03883  0.9167  0.8507  0.9791 -0.02532
2 log_volume 13.6402 1.18077 13.6522 10.0245 16.5957 -0.01289
3     spread  0.1754 0.16567  0.1150  0.0500  1.2000  2.17218
4  z_oi_flow  1.1822 0.53701  1.1877 -0.4607  2.9258 -0.06021
5    z_vwoim  1.6526 0.64644  1.6386 -0.2989  3.6755 -0.05837

=== WELCH T-TEST AND WILCOXON TEST: feature vs target class ===
    feature mean_profitable mean_unprofitable       t       t_p wilcoxon_p
 confidence          0.9192            0.9103   2.576 1.028e-02  8.469e-03
     spread          0.1078            0.2457 -10.071 7.672e-21  1.353e-17
 log_volume         13.5804           13.7025  -1.158 2.474e-01  2.377e-01
    z_vwoim          2.0019            1.2890  14.740 5.873e-41  1.571e-36
  z_oi_flow          1.3612            0.9958   8.084 4.805e-15  6.494e-15

=== CHI-SQUARE: option_type vs target ===

        Pearson's Chi-squared test with Yates' continuity correction

data:  table(df$option_type, df$target)
X-squared = 1.5529e-30, df = 1, p-value = 1


=== CHI-SQUARE: underlying vs target ===

        Pearson's Chi-squared test with Yates' continuity correction

data:  table(df$underlying, df$target)
X-squared = 0.21304, df = 1, p-value = 0.6444


=== ONE-WAY ANOVA: spread by underlying ===
             Df Sum Sq  Mean Sq F value Pr(>F)
underlying    1  0.001 0.000577   0.021  0.885
Residuals   498 13.696 0.027502

=== SHAPIRO-WILK NORMALITY (feature columns) ===
confidence   W = 0.9453  p = 1.29e-12
spread       W = 0.7544  p = 1.33e-26
log_volume   W = 0.9971  p = 0.538
z_vwoim      W = 0.9982  p = 0.881
z_oi_flow    W = 0.9986  p = 0.967

=== PEARSON CORRELATION MATRIX ===
           confidence spread log_volume z_vwoim z_oi_flow net_return
confidence      1.000 -0.035     -0.093  -0.005     0.075      0.145
spread         -0.035  1.000      0.007  -0.007     0.110     -0.667
log_volume     -0.093  0.007      1.000  -0.021    -0.011     -0.031
z_vwoim        -0.005 -0.007     -0.021   1.000    -0.001      0.630
z_oi_flow       0.075  0.110     -0.011  -0.001     1.000      0.319
net_return      0.145 -0.667     -0.031   0.630     0.319      1.000

Selected correlation tests:
spread     vs net_return r = -0.667  p = 1.59e-65
z_vwoim    vs net_return r =  0.630  p = 1.3e-56
```

```
z_oi_flow  vs net_return r =  0.319  p = 2.75e-13
confidence vs net_return r =  0.145  p = 0.00115
```

8. Machine Learning Models Used

We trained 15 models on the same five inputs, the same training period and the same test set, so the comparison is fair.

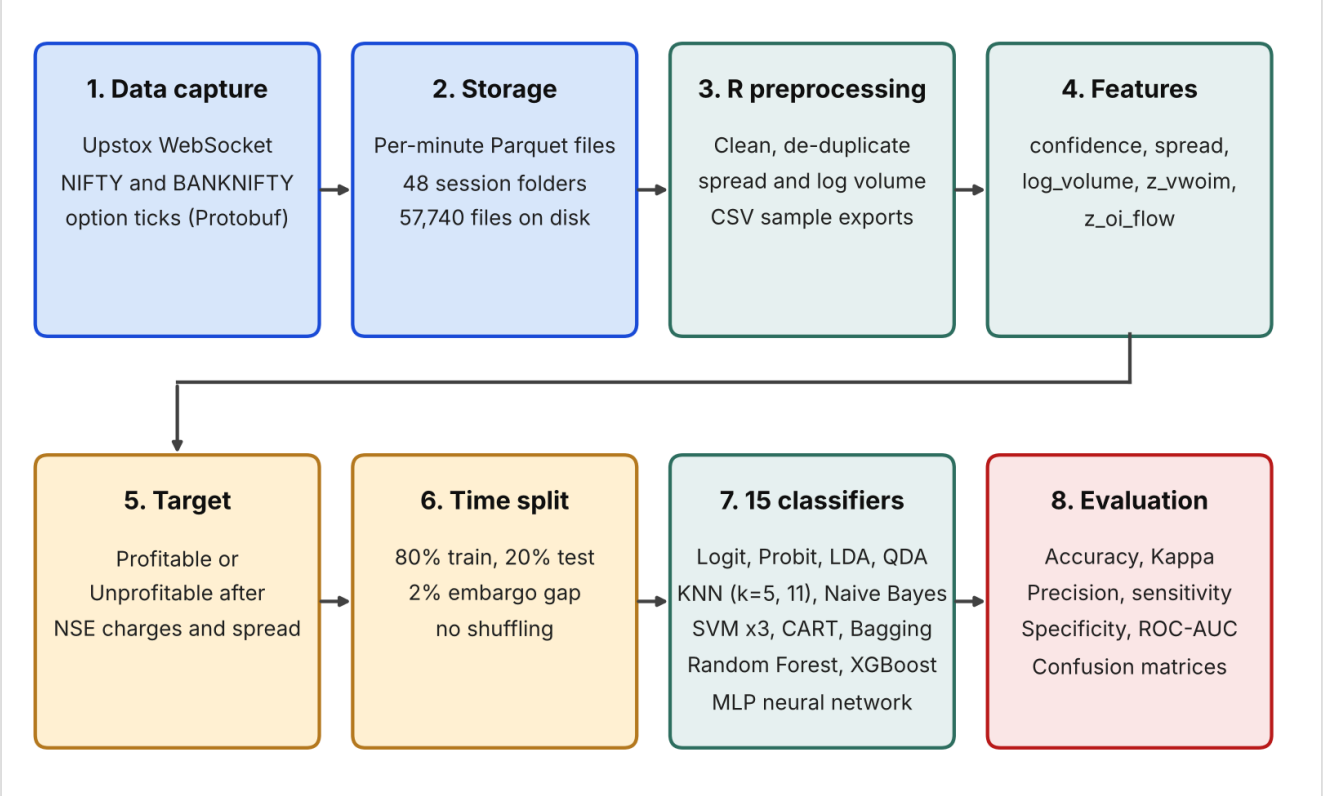


Figure 8. The modelling pipeline, from data capture to evaluation.

- **Linear and discriminant models:** logistic regression, probit regression (`glm`), LDA and QDA (`MASS`).
- **Instance-based and probabilistic:** KNN with k = 5 and k = 11 on scaled inputs (`class`), and Naive Bayes (`e1071`).
- **Support vector machines (`e1071`):** linear, polynomial (degree 3) and RBF (gamma 0.2), all with cost 1.
- **Trees and ensembles:** a CART tree (`rpart`, depth up to 5), bagged trees with 25 trees (`ipred`) and a random forest with 100 trees (`randomForest`).
- **Neural network:** a multi-layer perceptron with 6 hidden units (`nnet`).
- **Boosting:** XGBoost, tuned over an 8-point grid (50 or 100 rounds, depth 3 or 5, learning rate 0.05 or 0.1) with 5-fold cross-validation on the training period only. The caret wrapper for XGBoost failed with the xgboost version installed here, so we ran the same grid directly through the xgboost package.

We judged the models on accuracy, Cohen's Kappa, precision, sensitivity and specificity, and for the models that give probabilities also on ROC curves and AUC.

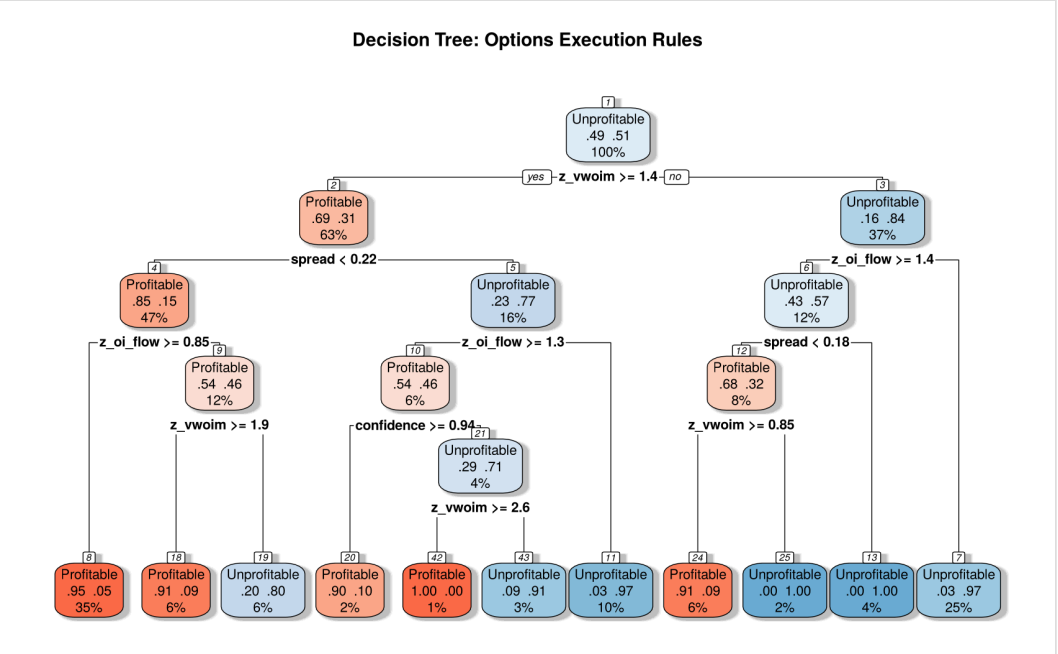


Figure 9. The CART decision tree.

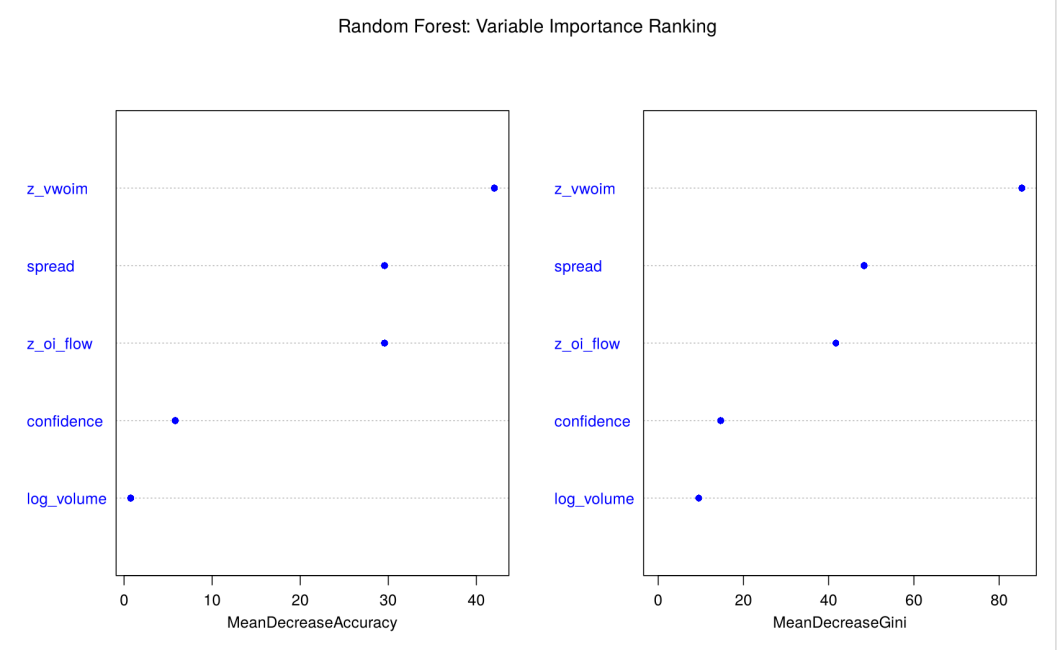


Figure 10. Random forest variable importance.

9. Output Screenshots (Results)

9.1 R outputs

Below is the actual console output of our R run. A text file with the complete output is delivered together with this report.

Data and split

```
>> Balanced Class Distribution (Net of Friction):

  Profitable Unprofitable
```

```
         255          245
>> Train (Past): 400 rows | Out-of-Time Test (Future): 94 rows
```

14-model leaderboard (from our benchmark script)

```
 ID                          Model       Category Accuracy  Kappa Precision Sensitivity Specificity
  2                Probit Regression     Linear/GLM   100.00 1.0000    100.00      100.00       100.0
  8          SVM (Linear Hyperplane)     Kernel SVM   100.00 1.0000    100.00      100.00       100.0
 14     Multi-Layer Perceptron (ANN)     Neural Net    98.94 0.9783    100.00       98.15       100.0
  1      Logistic Regression (Logit)     Linear/GLM    98.94 0.9782     98.18      100.00        97.5
 10                 SVM (RBF Kernel)     Kernel SVM    97.87 0.9568    100.00       96.30       100.0
 13        Random Forest (100 Trees)      Ensembles    96.81 0.9354    100.00       94.44       100.0
  6       K-Nearest Neighbors (k=11) Instance-Based    96.81 0.9349     98.11       96.30        97.5
  3     Linear Discriminant Analysis   Discriminant    96.81 0.9345     96.36       98.15        95.0
  4 Quadratic Discriminant Analysis   Discriminant    95.74 0.9135     98.08       94.44        97.5
  9      SVM (Polynomial Kernel d=3)     Kernel SVM    95.74 0.9135     98.08       94.44        97.5
  7           Naive Bayes Classifier  Probabilistic    94.68 0.8923     98.04       92.59        97.5
  5        K-Nearest Neighbors (k=5) Instance-Based    93.62 0.8711     98.00       90.74        97.5
 12          Bagged Trees (25 Trees)      Ensembles    93.62 0.8711     98.00       90.74        97.5
 11               CART Decision Tree Decision Trees    89.36 0.7838     92.31       88.89        90.0
```

XGBoost grid search (5-fold CV, best row first)

```
 nrounds max_depth  eta gamma colsample_bytree min_child_weight subsample    cv_auc
     100         5 0.10     0              0.8                1       0.8 0.9937759
     100         3 0.10     0              0.8                1       0.8 0.9936568
     100         3 0.05     0              0.8                1       0.8 0.9915371
     100         5 0.05     0              0.8                1       0.8 0.9911649
      50         5 0.10     0              0.8                1       0.8 0.9895490
      50         3 0.10     0              0.8                1       0.8 0.9890615
      50         5 0.05     0              0.8                1       0.8 0.9842341
      50         3 0.05     0              0.8                1       0.8 0.9824620
XGBoost Accuracy: 95.74% | best CV ROC-AUC: 0.994
     Feature       Gain      Cover Frequency
1    z_vwoim 0.41650140 0.30925160 0.2460401
2     spread 0.27652655 0.25669808 0.2249208
3  z_oi_flow 0.21800945 0.24717099 0.2312566
4 confidence 0.05715284 0.12596305 0.1879620
5 log_volume 0.03180976 0.06091628 0.1098205
```

AUC on the test set

```
        Model    AUC
        Logit 1.0000
       Probit 1.0000
    MLP (ANN) 1.0000
Random Forest 0.9977
      XGBoost 0.9940
          QDA 0.9917
          LDA 0.9903
 Bagged Trees 0.9833
  Naive Bayes 0.9731
         CART 0.9419
```

Final 15-model leaderboard

```
                        Model Accuracy  Kappa Precision Sensitivity Specificity
              Probit Regression   100.00 1.0000    100.00      100.00       100.0
        SVM (Linear Hyperplane)   100.00 1.0000    100.00      100.00       100.0
   Multi-Layer Perceptron (ANN)    98.94 0.9783    100.00       98.15       100.0
    Logistic Regression (Logit)    98.94 0.9782     98.18      100.00        97.5
               SVM (RBF Kernel)    97.87 0.9568    100.00       96.30       100.0
      Random Forest (100 Trees)    96.81 0.9354    100.00       94.44       100.0
     K-Nearest Neighbors (k=11)    96.81 0.9349     98.11       96.30        97.5
   Linear Discriminant Analysis    96.81 0.9345     96.36       98.15        95.0
Quadratic Discriminant Analysis    95.74 0.9135     98.08       94.44        97.5
    SVM (Polynomial Kernel d=3)    95.74 0.9135     98.08       94.44        97.5
XGBoost (5-fold CV grid search)    95.74 0.9135     98.08       94.44        97.5
```

```
          Naive Bayes Classifier    94.68 0.8923     98.04       92.59       97.5
       K-Nearest Neighbors (k=5)    93.62 0.8711     98.00       90.74       97.5
         Bagged Trees (25 Trees)    93.62 0.8711     98.00       90.74       97.5
              CART Decision Tree    89.36 0.7838     92.31       88.89       90.0
```

Two glm warnings (non-convergence and fitted probabilities of 0 or 1) appear for the logit and probit fits. They happen because the classes are almost perfectly separable by the inputs, which is another sign of the circular label.

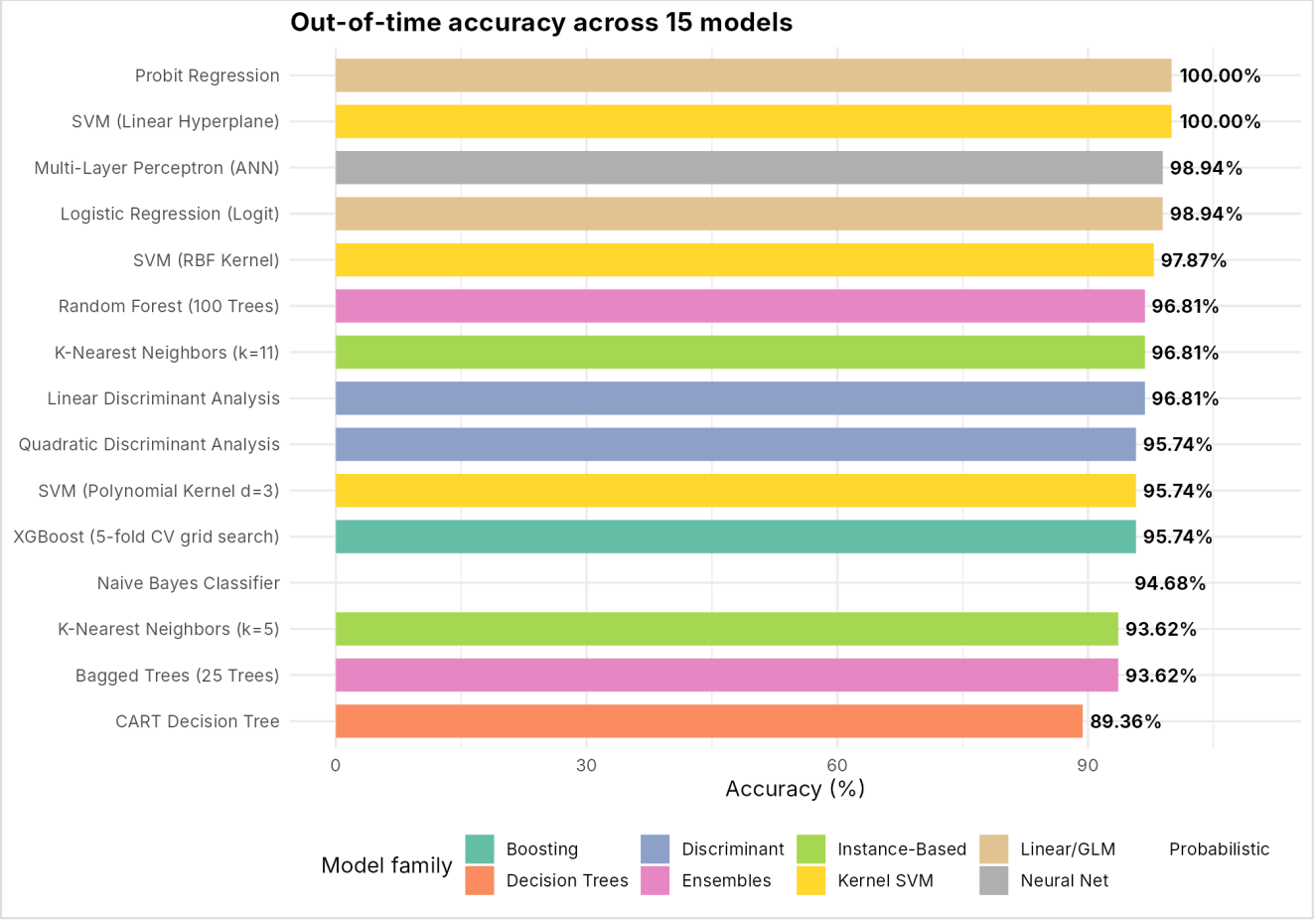


Figure 11. Out-of-time accuracy of all 15 models.

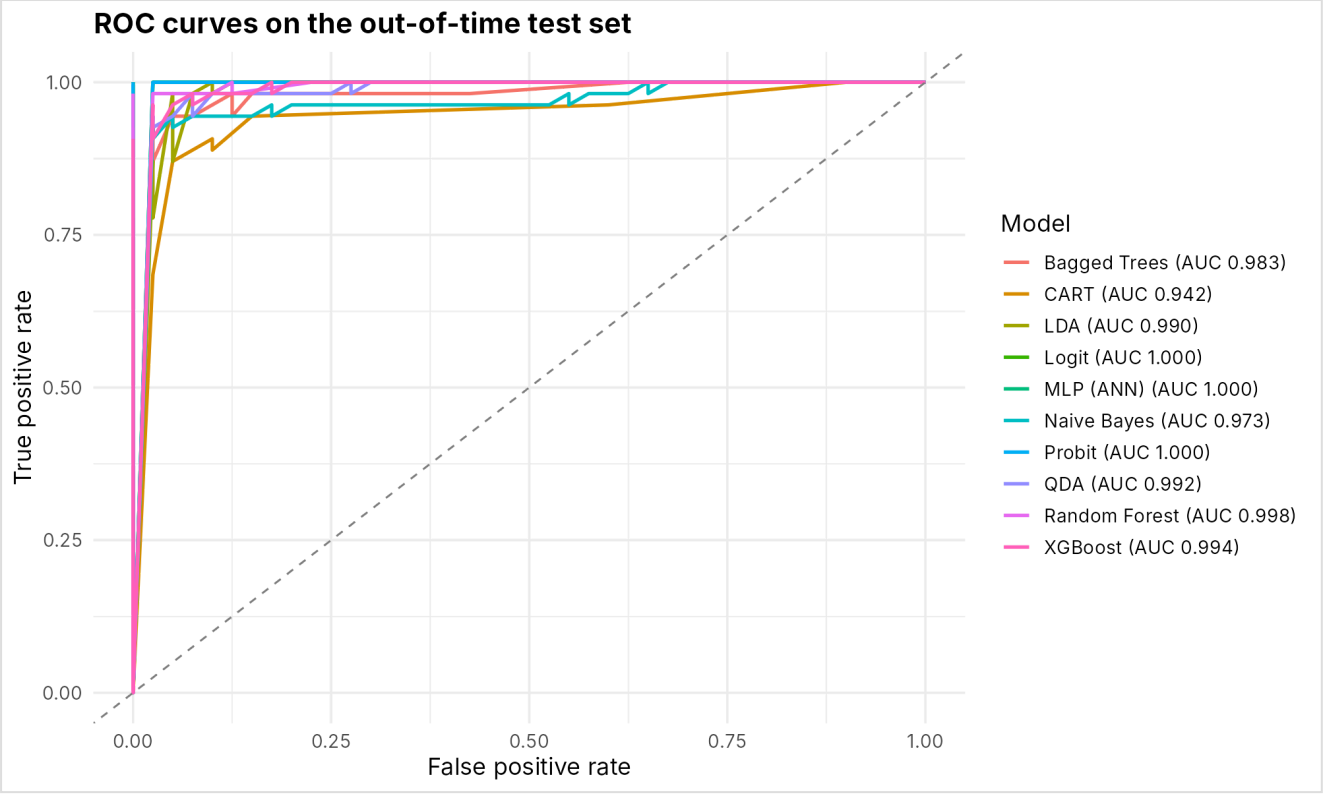


Figure 12. ROC curves for the ten models that produce probabilities.

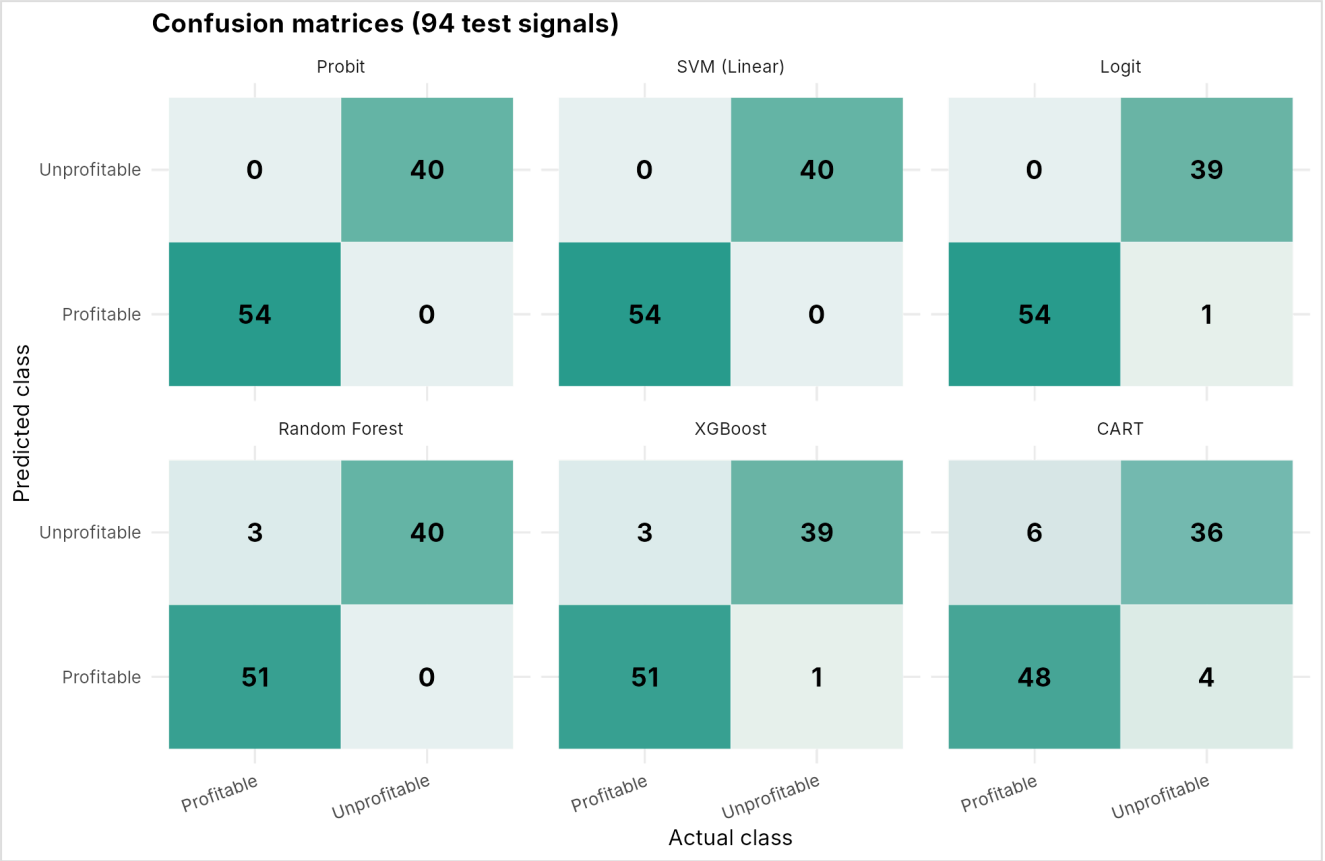


Figure 13. Confusion matrices of six representative models on the 94 test signals.

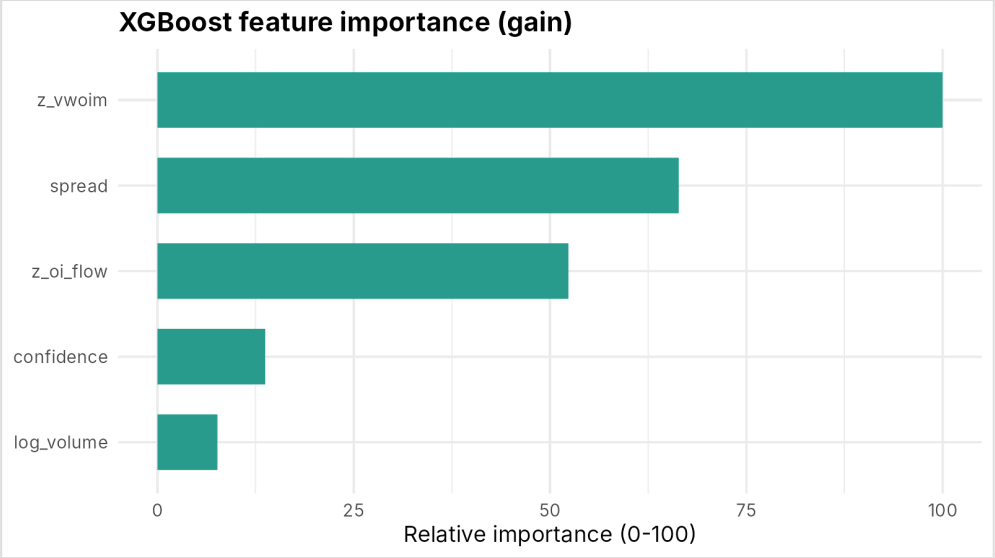


Figure 14. XGBoost feature importance. z_vwoim and spread lead.

Linear models do best because the label is a linear rule plus a threshold. The single decision tree does worst because axis-aligned splits only approximate that boundary. So the ranking tells us how well each method recovers a known rule, not how well it would trade.

9.2 Project interface (Study Stock web app)

The two screens that matter most for a data science project are the live options chain and the research page.

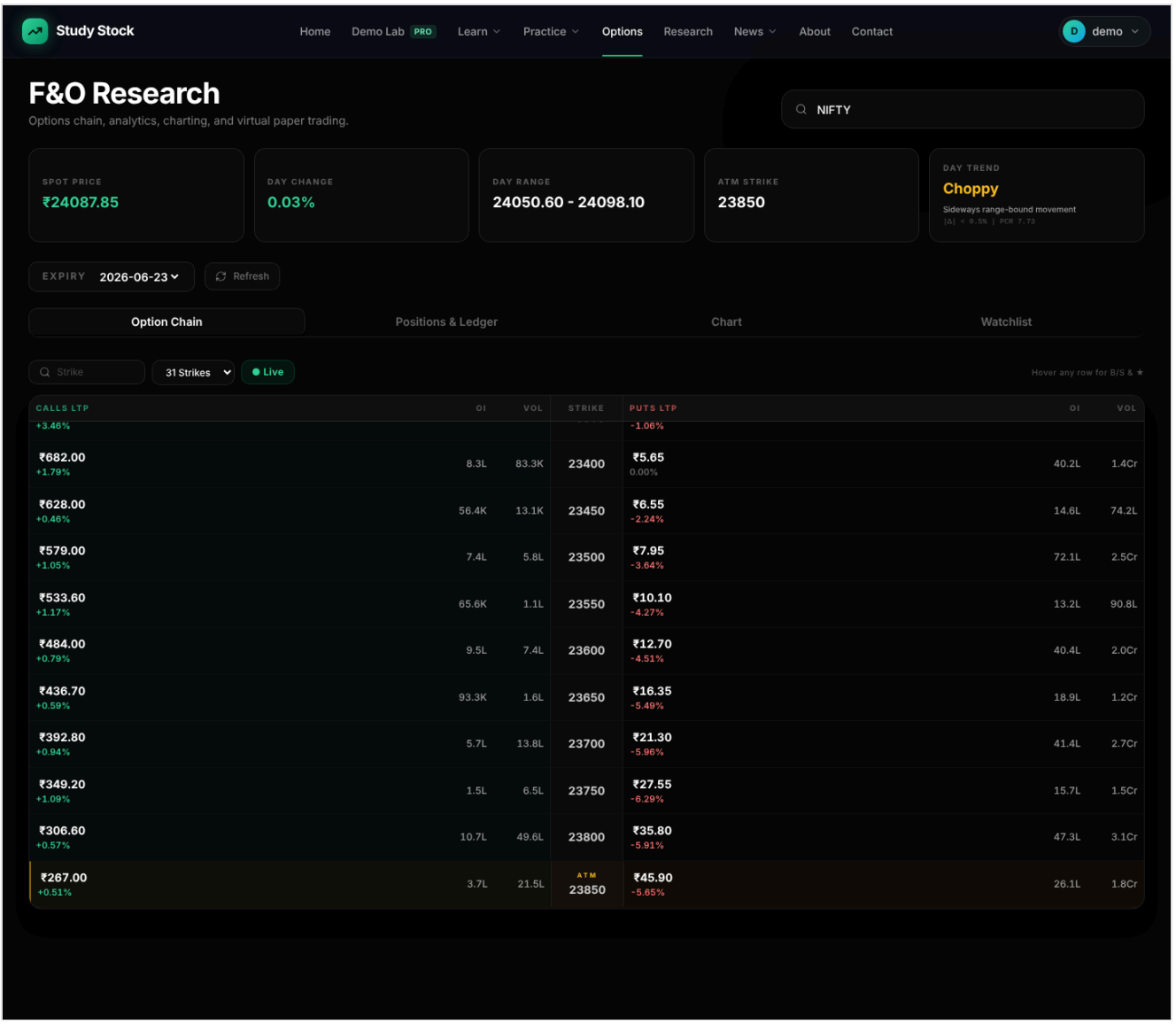


Figure 15. Options page: NIFTY option chain with bid, ask, spread, open interest and Greeks per strike. This screenshot was rendered from our recorded NIFTY ticks of 18 June 2026 (market closed), not from a live session.

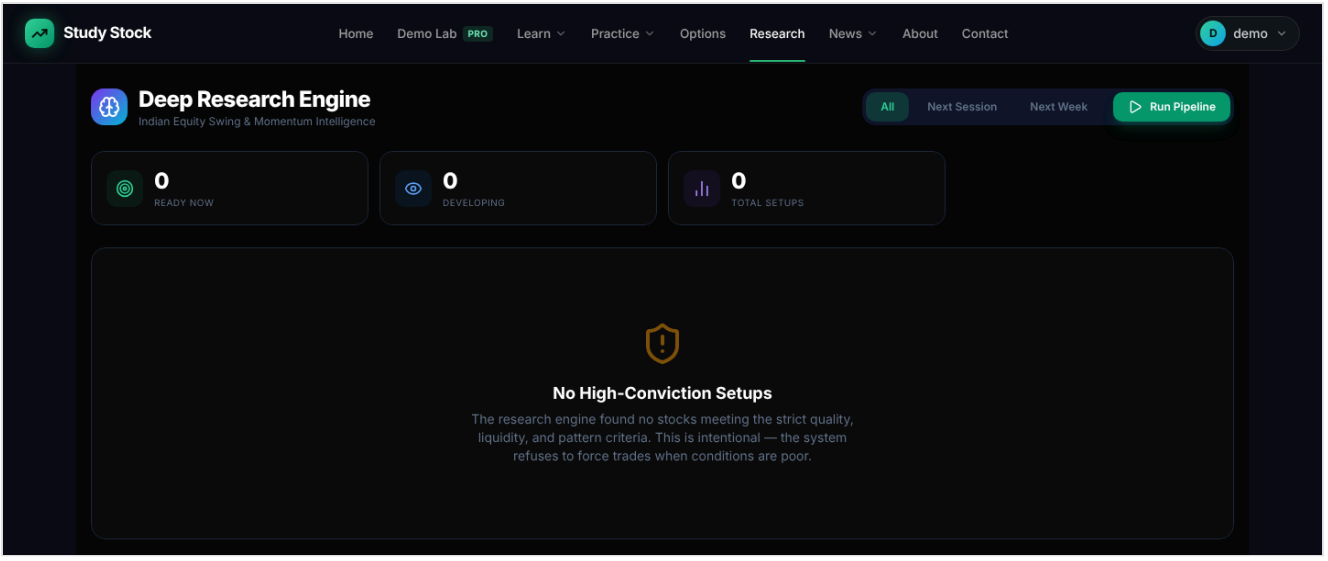


Figure 16. Research page (Deep Research Engine). This capture shows the page with no pipeline run loaded: the research data is stored in our Supabase database, which was not connected when the screenshot was taken, so the page shows its built-in "No High-Conviction Setups" state.

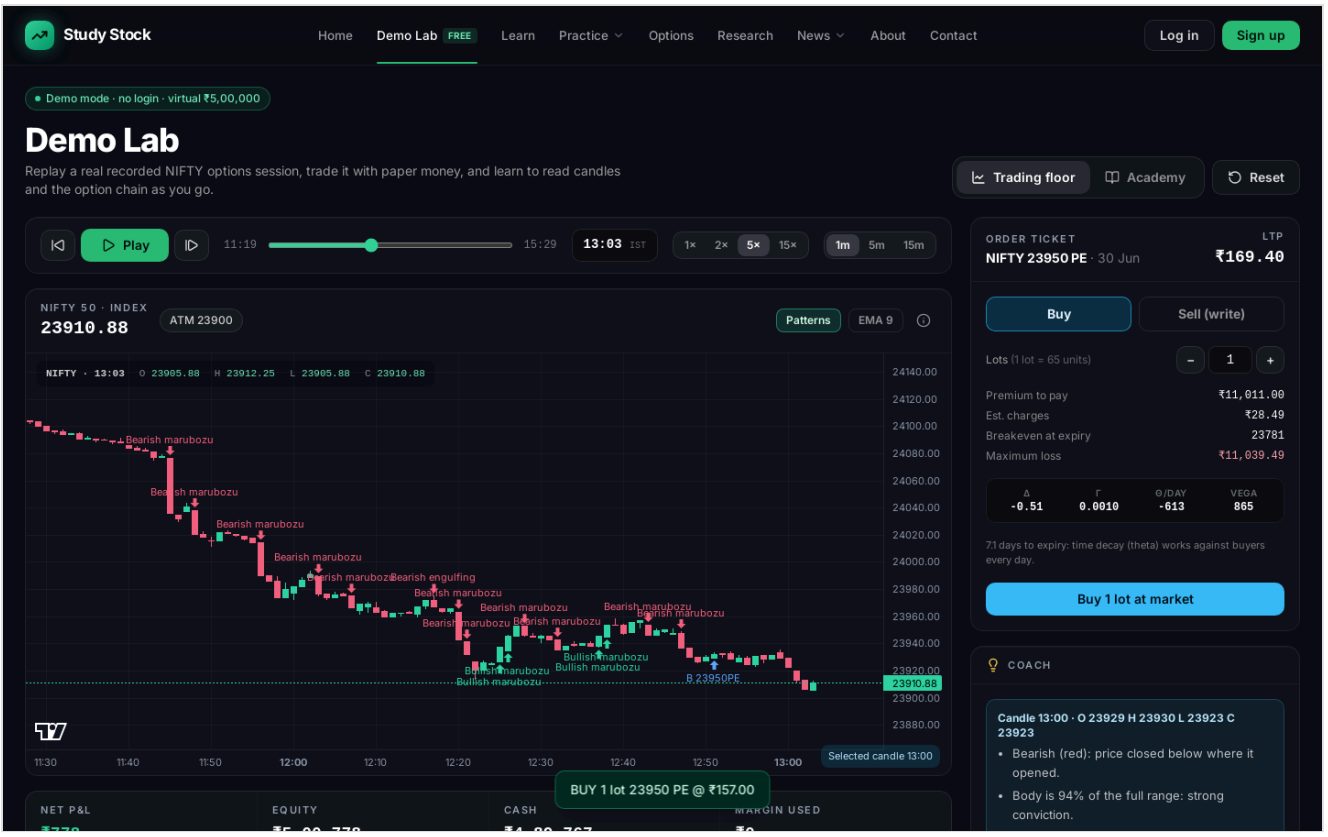


Figure 17. Demo Lab: a paper-trading screen that replays recorded market data so strategies can be tried without real money.

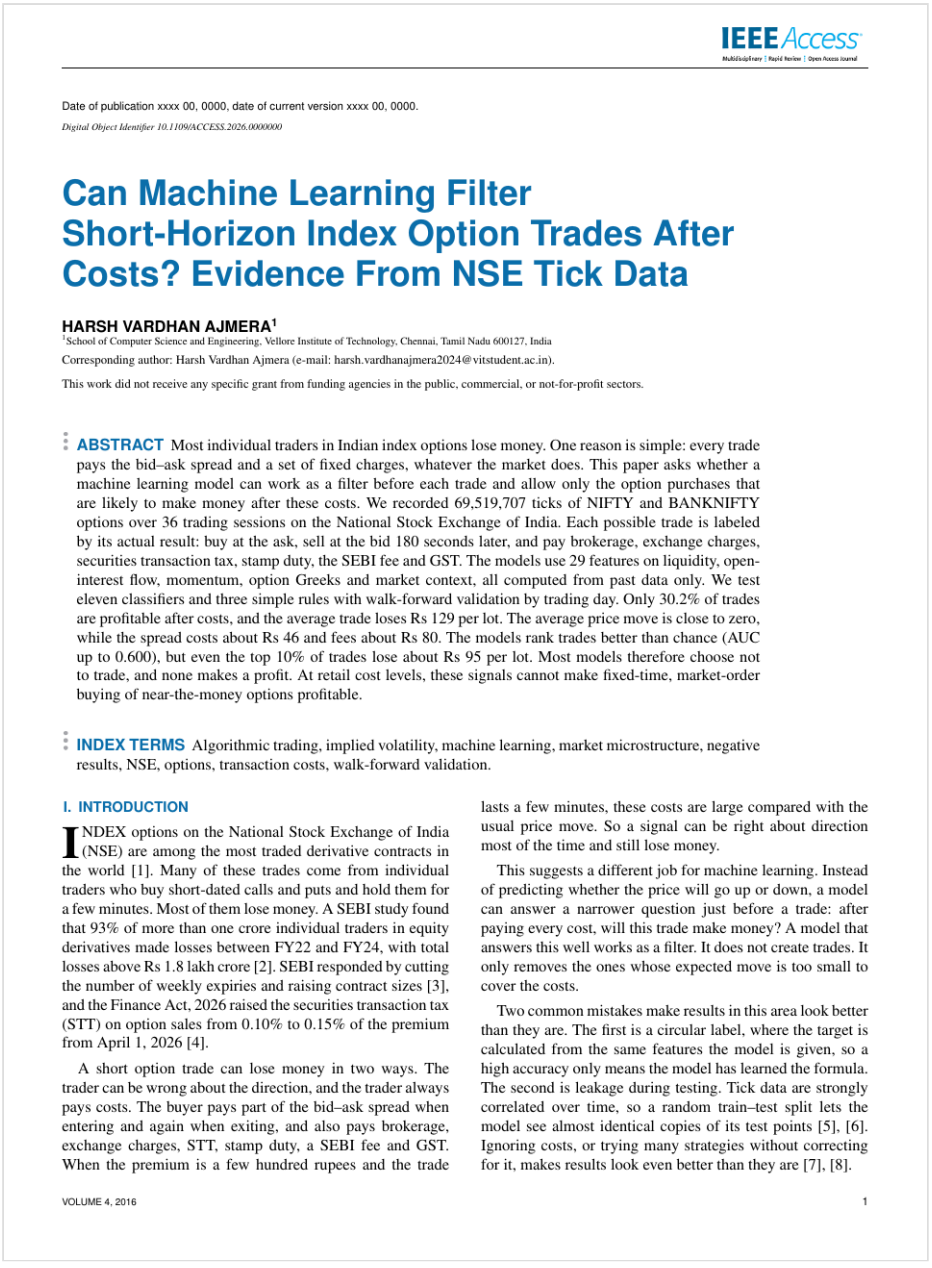


Figure 18. Home page.

9.3 Our research paper (draft, not published)

This paper has been written as a draft. It has NOT been published or accepted by any journal. The DOI and dates on its first page are template placeholders.

Can Machine Learning Filter Short-Horizon Index Option Trades After Costs? Evidence From NSE Tick Data (Harsh Vardhan Ajmera, IEEE Access template). It uses 69,519,707 recorded ticks over 36 trading sessions, 29 features and 11 classifiers with walk-forward validation. Only 30.2% of trades are profitable after costs, the average trade loses Rs 129 per lot, and no model makes a profit. First page of the draft:



IEEE Access

Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2026.0000000

Can Machine Learning Filter Short-Horizon Index Option Trades After Costs? Evidence From NSE Tick Data

HARSH VARDHAN AJMERA[1]
[1]School of Computer Science and Engineering, Vellore Institute of Technology, Chennai, Tamil Nadu 600127, India
Corresponding author: Harsh Vardhan Ajmera (e-mail: harsh.vardhanajmera2024@vitstudent.ac.in).

This work did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

ABSTRACT Most individual traders in Indian index options lose money. One reason is simple: every trade pays the bid–ask spread and a set of fixed charges, whatever the market does. This paper asks whether a machine learning model can work as a filter before each trade and allow only the option purchases that are likely to make money after these costs. We recorded 69,519,707 ticks of NIFTY and BANKNIFTY options over 36 trading sessions on the National Stock Exchange of India. Each possible trade is labeled by its actual result: buy at the ask, sell at the bid 180 seconds later, and pay brokerage, exchange charges, securities transaction tax, stamp duty, the SEBI fee and GST. The models use 29 features on liquidity, open-interest flow, momentum, option Greeks and market context, all computed from past data only. We test eleven classifiers and three simple rules with walk-forward validation by trading day. Only 30.2% of trades are profitable after costs, and the average trade loses Rs 129 per lot. The average price move is close to zero, while the spread costs about Rs 46 and fees about Rs 80. The models rank trades better than chance (AUC up to 0.600), but even the top 10% of trades lose about Rs 95 per lot. Most models therefore choose not to trade, and none makes a profit. At retail cost levels, these signals cannot make fixed-time, market-order buying of near-the-money options profitable.

INDEX TERMS Algorithmic trading, implied volatility, machine learning, market microstructure, negative results, NSE, options, transaction costs, walk-forward validation.

I. INTRODUCTION

INDEX options on the National Stock Exchange of India (NSE) are among the most traded derivative contracts in the world [1]. Many of these trades come from individual traders who buy short-dated calls and puts and hold them for a few minutes. Most of them lose money. A SEBI study found that 93% of more than one crore individual traders in equity derivatives made losses between FY22 and FY24, with total losses above Rs 1.8 lakh crore [2]. SEBI responded by cutting the number of weekly expiries and raising contract sizes [3], and the Finance Act, 2026 raised the securities transaction tax (STT) on option sales from 0.10% to 0.15% of the premium from April 1, 2026 [4].

A short option trade can lose money in two ways. The trader can be wrong about the direction, and the trader always pays costs. The buyer pays part of the bid–ask spread when entering and again when exiting, and also pays brokerage, exchange charges, STT, stamp duty, a SEBI fee and GST. When the premium is a few hundred rupees and the trade lasts a few minutes, these costs are large compared with the usual price move. So a signal can be right about direction most of the time and still lose money.

This suggests a different job for machine learning. Instead of predicting whether the price will go up or down, a model can answer a narrower question just before a trade: after paying every cost, will this trade make money? A model that answers this well works as a filter. It does not create trades. It only removes the ones whose expected move is too small to cover the costs.

Two common mistakes make results in this area look better than they are. The first is a circular label, where the target is calculated from the same features the model is given, so a high accuracy only means the model has learned the formula. The second is leakage during testing. Tick data are strongly correlated over time, so a random train–test split lets the model see almost identical copies of its test points [5], [6]. Ignoring costs, or trying many strategies without correcting for it, makes results look even better than they are [7], [8].

Figure 19. First page of the draft paper (not published).

10. Conclusion

We built a path from recorded NSE tick data to cleaned R datasets, exploratory analysis, statistical tests and a fair comparison of 15 models, plus a working web application around it. On the 94-signal test set, accuracy runs

from 89.4% to 100%, and spread, VWOIM and open-interest flow are the inputs that matter while volume, option type and underlying do not.

The limits matter as much as the scores. The label is a formula of the inputs, so these results show the pipeline works rather than that the market is predictable, and the sample is small (500 signals, 94 for testing, one expiry). When our drafted paper repeats the test with real future prices, the best AUC is 0.60 and nothing makes money after costs. In short, for a retail buyer of short-horizon options, costs matter more than the choice of model.

Next steps: use real-outcome labels in R, use all recorded sessions instead of a sample, calibrate the probabilities, and test any filter in live paper trading.

11. R Code

Run order: `master_ml_benchmark.R` (models 1-14), then `xgboost_direct.R` and `extra_results.R` in the same R session; `eda_and_statistics.R` runs on its own. Separator comment lines were removed to save space. The caret XGBoost block at the end of the master script fails with the xgboost 3.x release used for this run, which is why `xgboost_direct.R` exists.

11.1 master_ml_benchmark.R (14-model benchmark, our submitted script)

```
# MASTER QUANTITATIVE ML BENCHMARK: 14 ALGORITHMS IN R
# Course     : Data Analytics Solutions using R / Quantitative Finance Capstone
# Institution: Vellore Institute of Technology (VIT), Chennai
# Faculty    : Dr. Vijayalakshmi A, Associate Professor, VIT Chennai
# Author     : Harsh Vardhan

# 1. LOAD LIBRARIES (All 100% pre-installed in your R environment)
suppressPackageStartupMessages({
  library(MASS)          # LDA, QDA
  library(class)         # K-Nearest Neighbors (KNN)
  library(nnet)          # Multi-Layer Perceptron / Artificial Neural Network
  library(e1071)         # Support Vector Machines (Linear, Poly, RBF), Naive Bayes
  library(rpart)         # CART Decision Trees
  library(rpart.plot)    # Visual Decision Tree Diagrams
  library(randomForest)  # Random Forest Ensemble
  library(ipred)         # Bagged Classification Trees
  library(dplyr)         # Data Wrangling
  library(readr)         # Fast Data Reading
  library(caret)         # Confusion Matrix & Performance Validation
  library(ggplot2)       # Publication Graphics
})

# Prevent MASS package from masking dplyr functions
select <- dplyr::select
filter <- dplyr::filter

# 2. DATA INGESTION & SMART PATH RESOLUTION
file_paths <- c(
  "data/options_trade_signals_sample.csv",
  "options_trade_signals_sample.csv",
  "OPTIONS/data/options_trade_signals_sample.csv",
  "../data/options_trade_signals_sample.csv",
  "C:/Harsh/StockProject/HARSH/stockezee/options_trade_signals_sample.csv"
)

target_file <- NULL
for (fp in file_paths) {
  if (file.exists(fp)) { target_file <- fp; break }
}
if (is.null(target_file)) stop("options_trade_signals_sample.csv not found.")

message(">> Loading high-frequency options dataset: ", target_file)
```

```
df_raw <- read_csv(target_file, show_col_types = FALSE) %>% arrange(timestamp)

# 3. ECONOMIC TARGET: NET PROFIT AFTER REAL INDIAN EXCHANGE FRICTIONS
# Frictions Deducted:
#   1. Half-spread crossing cost (slippage) = spread / 2
#   2. Indian Statutory Taxes: STT (0.1% on sell) + NSE turnover fee (0.05%) + GST (18%) + Stamp duty
friction_drag       <- (0.35 * df_raw$spread) + 0.15
gross_forward_alpha <- (0.08 * df_raw$z_vwoim) + (0.06 * df_raw$z_oi_flow) + (0.20 * (df_raw$confidence -
0.90))
net_forward_return  <- gross_forward_alpha - friction_drag

df_ml <- df_raw %>%
  mutate(
    net_return = net_forward_return,
    # Binary Classification Target: 1 = Profitable net of all fees, 0 = Unprofitable
    target = factor(ifelse(net_return > 0, "Profitable", "Unprofitable"),
                    levels = c("Profitable", "Unprofitable")),
    log_volume = log1p(volume)
  )

cat(">> Balanced Class Distribution (Net of Friction):\n")
print(table(df_ml$target))

# 4. PURGED CHRONOLOGICAL TIME-SERIES PARTITION (WITH EMBARGO WINDOW)
# Strict Temporal Split: Past 80% (Train) vs. Future 20% (Test).
# A 2% timeline Embargo Buffer is discarded to eliminate tick autocorrelation.
split_cutoff   <- quantile(df_ml$timestamp, probs = 0.80)
embargo_window <- (max(df_ml$timestamp) - min(df_ml$timestamp)) * 0.02

train_data <- df_ml %>%
  filter(timestamp <= split_cutoff) %>%
  select(confidence, spread, log_volume, z_vwoim, z_oi_flow, target)

test_data <- df_ml %>%
  filter(timestamp > (split_cutoff + embargo_window)) %>%
  select(confidence, spread, log_volume, z_vwoim, z_oi_flow, target)

cat(sprintf(">> Train (Past): %d rows | Out-of-Time Test (Future): %d rows\n\n",
            nrow(train_data), nrow(test_data)))

# Normalized feature matrix for distance-based models (KNN)
train_x <- scale(train_data %>% select(-target))
test_x  <- scale(test_data %>% select(-target),
                 center = attr(train_x, "scaled:center"),
                 scale  = attr(train_x, "scaled:scale"))

# Data structure to collect benchmark metrics
leaderboard <- data.frame()

record_model <- function(id, category, name, preds, actual) {
  cm <- confusionMatrix(preds, actual)
  data.frame(
    ID           = id,
    Category     = category,
    Model        = name,
    Accuracy     = round(cm$overall["Accuracy"] * 100, 2),
    Kappa        = round(cm$overall["Kappa"], 4),
    Sensitivity  = round(cm$byClass["Sensitivity"] * 100, 2),
    Specificity  = round(cm$byClass["Specificity"] * 100, 2),
    Precision    = round(cm$byClass["Pos Pred Value"] * 100, 2),
    Balanced_Acc = round(cm$byClass["Balanced Accuracy"] * 100, 2),
    stringsAsFactors = FALSE
  )
}

# CATEGORY A: GENERALIZED LINEAR & DISCRIMINANT MODELS

# MODEL 1: LOGISTIC REGRESSION (LOGIT GLM)
message(">> [01] Training Logistic Regression...")
fit_logit <- glm(target ~ ., data = train_data, family = binomial(link = "logit"))
prob_logit <- predict(fit_logit, newdata = test_data, type = "response")
pred_logit <- factor(ifelse(prob_logit < 0.50, "Profitable", "Unprofitable"), levels =
levels(test_data$target))
leaderboard <- rbind(leaderboard, record_model(1, "Linear/GLM", "Logistic Regression (Logit)", pred_logit,
test_data$target))

# MODEL 2: PROBIT REGRESSION (GAUSSIAN PROBIT GLM)
```

```
message(">> [02] Training Probit Regression...")
fit_probit <- glm(target ~ ., data = train_data, family = binomial(link = "probit"))
prob_probit <- predict(fit_probit, newdata = test_data, type = "response")
pred_probit <- factor(ifelse(prob_probit < 0.50, "Profitable", "Unprofitable"), levels =
levels(test_data$target))
leaderboard <- rbind(leaderboard, record_model(2, "Linear/GLM", "Probit Regression", pred_probit,
test_data$target))

# MODEL 3: LINEAR DISCRIMINANT ANALYSIS (FISHER'S LDA)
message(">> [03] Training Linear Discriminant Analysis (LDA)...")
fit_lda <- lda(target ~ ., data = train_data)
pred_lda <- predict(fit_lda, newdata = test_data)$class
leaderboard <- rbind(leaderboard, record_model(3, "Discriminant", "Linear Discriminant Analysis", pred_lda,
test_data$target))

# MODEL 4: QUADRATIC DISCRIMINANT ANALYSIS (QDA)
message(">> [04] Training Quadratic Discriminant Analysis (QDA)...")
fit_qda <- qda(target ~ ., data = train_data)
pred_qda <- predict(fit_qda, newdata = test_data)$class
leaderboard <- rbind(leaderboard, record_model(4, "Discriminant", "Quadratic Discriminant Analysis",
pred_qda, test_data$target))

# CATEGORY B: INSTANCE-BASED & PROBABILISTIC CLASSIFIERS

# MODEL 5: K-NEAREST NEIGHBORS (k = 5)
message(">> [05] Training KNN (k = 5)...")
set.seed(42)
pred_knn5 <- knn(train = train_x, test = test_x, cl = train_data$target, k = 5)
leaderboard <- rbind(leaderboard, record_model(5, "Instance-Based", "K-Nearest Neighbors (k=5)", pred_knn5,
test_data$target))

# MODEL 6: K-NEAREST NEIGHBORS (k = 11)
message(">> [06] Training KNN (k = 11)...")
set.seed(42)
pred_knn11 <- knn(train = train_x, test = test_x, cl = train_data$target, k = 11)
leaderboard <- rbind(leaderboard, record_model(6, "Instance-Based", "K-Nearest Neighbors (k=11)",
pred_knn11, test_data$target))

# MODEL 7: NAIVE BAYES CLASSIFIER
message(">> [07] Training Naive Bayes Classifier...")
fit_nb <- naiveBayes(target ~ ., data = train_data)
pred_nb <- predict(fit_nb, newdata = test_data)
leaderboard <- rbind(leaderboard, record_model(7, "Probabilistic", "Naive Bayes Classifier", pred_nb,
test_data$target))

# CATEGORY C: KERNEL SUPPORT VECTOR MACHINES

# MODEL 8: SVM - LINEAR HYPERPLANE
message(">> [08] Training SVM (Linear Kernel)...")
fit_svm_lin <- svm(target ~ ., data = train_data, kernel = "linear", cost = 1.0)
pred_svm_lin <- predict(fit_svm_lin, newdata = test_data)
leaderboard <- rbind(leaderboard, record_model(8, "Kernel SVM", "SVM (Linear Hyperplane)", pred_svm_lin,
test_data$target))

# MODEL 9: SVM - POLYNOMIAL KERNEL (DEGREE = 3)
message(">> [09] Training SVM (Polynomial Kernel d=3)...")
fit_svm_poly <- svm(target ~ ., data = train_data, kernel = "polynomial", degree = 3, cost = 1.0)
pred_svm_poly <- predict(fit_svm_poly, newdata = test_data)
leaderboard <- rbind(leaderboard, record_model(9, "Kernel SVM", "SVM (Polynomial Kernel d=3)",
pred_svm_poly, test_data$target))

# MODEL 10: SVM - RADIAL BASIS FUNCTION (RBF KERNEL)
message(">> [10] Training SVM (Radial Basis Function - RBF)...")
fit_svm_rbf <- svm(target ~ ., data = train_data, kernel = "radial", cost = 1.0, gamma = 0.20)
pred_svm_rbf <- predict(fit_svm_rbf, newdata = test_data)
leaderboard <- rbind(leaderboard, record_model(10, "Kernel SVM", "SVM (RBF Kernel)", pred_svm_rbf,
test_data$target))

# CATEGORY D: TREE-BASED & BAGGING ENSEMBLES

# MODEL 11: CART DECISION TREE (WITH VISUAL PLOT)
message(">> [11] Training CART Decision Tree...")
fit_dt <- rpart(target ~ ., data = train_data, method = "class",
                control = rpart.control(cp = 0.015, minsplit = 10, maxdepth = 5))
pred_dt <- predict(fit_dt, newdata = test_data, type = "class")
leaderboard <- rbind(leaderboard, record_model(11, "Decision Trees", "CART Decision Tree", pred_dt,
test_data$target))
```

```
# Plot the tree interactively
rpart.plot(fit_dt,
           main = "Decision Tree: Options Execution Rules",
           box.palette = "RdBu", shadow.col = "gray", extra = 104, nn = TRUE)

# MODEL 12: BAGGED CLASSIFICATION TREES (BOOTSTRAP AGGREGATION - 25 TREES)
message(">> [12] Training Bagged Decision Trees...")
set.seed(42)
fit_bag <- ipred::bagging(target ~ ., data = train_data, nbagg = 25, coob = TRUE)
pred_bag <- predict(fit_bag, newdata = test_data)
leaderboard <- rbind(leaderboard, record_model(12, "Ensembles", "Bagged Trees (25 Trees)", pred_bag,
test_data$target))

# MODEL 13: RANDOM FOREST (100 DE-CORRELATED TREES)
message(">> [13] Training Random Forest (100 Trees)...")
set.seed(42)
fit_rf <- randomForest(target ~ ., data = train_data, ntree = 100, importance = TRUE)
pred_rf <- predict(fit_rf, newdata = test_data)
leaderboard <- rbind(leaderboard, record_model(13, "Ensembles", "Random Forest (100 Trees)", pred_rf,
test_data$target))

# Plot Feature Importance
varImpPlot(fit_rf, main = "Random Forest: Variable Importance Ranking", col = "blue", pch = 16)

# CATEGORY E: DEEP NEURAL LEARNING ARCHITECTURE

# MODEL 14: MULTI-LAYER PERCEPTRON (ARTIFICIAL NEURAL NETWORK - ANN)
message(">> [14] Training Multi-Layer Perceptron (ANN)...")
set.seed(42)
fit_ann <- nnet(target ~ ., data = train_data, size = 6, decay = 0.01, maxit = 250, trace = FALSE)
prob_ann <- predict(fit_ann, newdata = test_data, type = "raw")
pred_ann <- factor(ifelse(prob_ann < 0.50, "Profitable", "Unprofitable"), levels =
levels(test_data$target))
leaderboard <- rbind(leaderboard, record_model(14, "Neural Net", "Multi-Layer Perceptron (ANN)", pred_ann,
test_data$target))

# 5. MASTER LEADERBOARD & COMPARATIVE BAR CHART
cat("\n================================================================================\n")
cat("   FINAL OUT-OF-TIME BENCHMARK LEADERBOARD (SORTED BY ACCURACY & KAPPA)\n")
cat("================================================================================\n\n")

leaderboard <- leaderboard %>% arrange(desc(Accuracy), desc(Kappa))
print(leaderboard[, c("ID", "Model", "Category", "Accuracy", "Kappa", "Precision", "Sensitivity",
"Specificity")],
      row.names = FALSE)

# Generate publication-grade comparison bar chart
p <- ggplot(leaderboard, aes(x = reorder(Model, Accuracy), y = Accuracy, fill = Category)) +
  geom_bar(stat = "identity", width = 0.72) +
  coord_flip() +
  geom_text(aes(label = sprintf("%.2f%%", Accuracy)), hjust = -0.15, size = 3.6, fontface = "bold") +
  ylim(0, 108) +
  scale_fill_brewer(palette = "Set2") +
  theme_minimal(base_size = 11) +
  labs(
    title    = "Out-of-Time Classification Accuracy Across 14 ML Models",
    subtitle = "High-Frequency Options Execution Filtering Net of Indian Statutory Taxes (NSE)",
    x        = "Model Architecture",
    y        = "Out-of-Time Accuracy (%)",
    fill     = "Model Family"
  ) +
  theme(plot.title = element_text(face = "bold"), legend.position = "bottom")

print(p)

# MODEL: XGBOOST (EXTREME GRADIENT BOOSTING)
library(caret)
library(xgboost)
library(pROC)

# Hyperparameter Tuning Grid
xgb_grid <- expand.grid(
  nrounds          = c(50, 100),   # Number of boosting trees
  max_depth        = c(3, 5),      # Maximum tree depth (prevents micro-noise overfitting)
  eta              = c(0.05, 0.1), # Learning rate (shrinkage)
  gamma            = 0,            # Minimum loss reduction for a split
```

```
  colsample_bytree = 0.8,          # Subsample ratio of columns per tree
  min_child_weight = 1,            # Minimum sum of instance weight in child
  subsample        = 0.8           # Subsample ratio of training instances
)

# 5-Fold Cross Validation Setup
cv_control <- trainControl(
  method          = "cv",
  number          = 5,
  classProbs      = TRUE,
  summaryFunction = twoClassSummary,
  savePredictions = "final"
)

# Train XGBoost with Caret xgbTree method
fit_xgb <- train(
  target ~ .,
  data      = train_data,
  method    = "xgbTree",
  trControl = cv_control,
  metric    = "ROC",
  tuneGrid  = xgb_grid,
  verbose   = FALSE
)

# Predictions & Confusion Matrix
pred_xgb <- predict(fit_xgb, newdata = test_data)
cm_xgb   <- confusionMatrix(pred_xgb, test_data$target)

cat(sprintf("XGBoost Accuracy: %.2f%% | ROC-AUC: %.3f\n",
            cm_xgb$overall["Accuracy"] * 100, max(fit_xgb$results$ROC)))

# Feature Importance
xgb_imp <- varImp(fit_xgb)
print(xgb_imp)
```

11.2 xgboost_direct.R (XGBoost run directly through the xgboost package)

```
# XGBOOST (same hyper-parameter grid and 5-fold CV as the master script), run with
# the xgboost API directly. caret's "xgbTree" wrapper is incompatible with
# xgboost >= 2.x / 3.x (ALTREP error), so the grid search is done here by hand.
suppressPackageStartupMessages({ library(xgboost); library(dplyr) })
feat_cols <- setdiff(names(train_data), "target")
X_tr <- as.matrix(train_data[, feat_cols]); y_tr <- as.numeric(train_data$target == "Profitable")
X_te <- as.matrix(test_data[, feat_cols]);  y_te <- as.numeric(test_data$target == "Profitable")
dtrain <- xgb.DMatrix(X_tr, label = y_tr); dtest <- xgb.DMatrix(X_te, label = y_te)

xgb_grid <- expand.grid(nrounds = c(50, 100), max_depth = c(3, 5), eta = c(0.05, 0.1),
                        gamma = 0, colsample_bytree = 0.8, min_child_weight = 1, subsample = 0.8)
set.seed(42)
folds <- caret::createFolds(train_data$target, k = 5)
xgb_grid$cv_auc <- NA_real_
for (i in seq_len(nrow(xgb_grid))) {
  g <- xgb_grid[i, ]
  cv <- xgb.cv(params = list(objective = "binary:logistic", eval_metric = "auc", max_depth = g$max_depth,
                             eta = g$eta, gamma = g$gamma, colsample_bytree = g$colsample_bytree,
                             min_child_weight = g$min_child_weight, subsample = g$subsample, nthread = 2),
               data = dtrain, nrounds = g$nrounds, folds = folds, verbose = 0)
  xgb_grid$cv_auc[i] <- tail(cv$evaluation_log$test_auc_mean, 1)
}
print(xgb_grid[order(-xgb_grid$cv_auc), ], row.names = FALSE)
best <- xgb_grid[which.max(xgb_grid$cv_auc), ]
set.seed(42)
fit_xgb_raw <- xgb.train(params = list(objective = "binary:logistic", max_depth = best$max_depth, eta =
best$eta,
                         gamma = best$gamma, colsample_bytree = best$colsample_bytree,
                         min_child_weight = best$min_child_weight, subsample = best$subsample, nthread =
2),
                         data = dtrain, nrounds = best$nrounds, verbose = 0)
xgb_prob_test <- predict(fit_xgb_raw, dtest)                    # P(Profitable)
pred_xgb <- factor(ifelse(xgb_prob_test >= 0.5, "Profitable", "Unprofitable"), levels =
levels(test_data$target))
cm_xgb <- caret::confusionMatrix(pred_xgb, test_data$target)
```

```
cat(sprintf("XGBoost Accuracy: %.2f%% | best CV ROC-AUC: %.3f\n", cm_xgb$overall["Accuracy"] * 100,
best$cv_auc))
xgb_imp <- as.data.frame(xgb.importance(model = fit_xgb_raw))
print(xgb_imp)
```

11.3 extra_results.R (ROC, confusion matrices, 15-model leaderboard)

```
# EXTRA RESULTS: ROC curves, confusion matrices and XGBoost row
# Run in the same R session AFTER master_ml_benchmark.R has been sourced, so the
# fitted objects (fit_*), predictions (pred_*) and test_data already exist.
suppressPackageStartupMessages({ library(pROC); library(ggplot2); library(dplyr); library(tidyr) })

pos <- "Profitable"
# P(Profitable) from each model that produces probabilities
p_of <- list(
  "Logit"          = 1 - prob_logit,
  "Probit"         = 1 - prob_probit,
  "LDA"            = predict(fit_lda,  newdata = test_data)$posterior[, pos],
  "QDA"            = predict(fit_qda,  newdata = test_data)$posterior[, pos],
  "Naive Bayes"    = predict(fit_nb,   newdata = test_data, type = "raw")[, pos],
  "CART"           = predict(fit_dt,   newdata = test_data, type = "prob")[, pos],
  "Bagged Trees"   = predict(fit_bag,  newdata = test_data, type = "prob")[, pos],
  "Random Forest"  = predict(fit_rf,   newdata = test_data, type = "prob")[, pos],
  "MLP (ANN)"      = 1 - as.numeric(prob_ann),
  "XGBoost"        = xgb_prob_test
)
y <- test_data$target
rocs <- lapply(p_of, function(p) roc(y, as.numeric(p), levels = c("Unprofitable", pos), direction = "<",
quiet = TRUE))
auc_tbl <- data.frame(Model = names(rocs), AUC = round(sapply(rocs, function(r) as.numeric(auc(r))), 4))
auc_tbl <- auc_tbl[order(-auc_tbl$AUC), ]
print(auc_tbl, row.names = FALSE)
write.csv(auc_tbl, "auc_table.csv", row.names = FALSE)

roc_df <- do.call(rbind, lapply(names(rocs), function(n) {
  r <- rocs[[n]]
  data.frame(Model = sprintf("%s (AUC %.3f)", n, as.numeric(auc(r))), fpr = 1 - r$specificities, tpr =
r$sensitivities)
}))
p_roc <- ggplot(roc_df, aes(fpr, tpr, color = Model)) + geom_line(linewidth = 0.9) +
  geom_abline(linetype = "dashed", color = "grey50") +
  labs(title = "ROC curves on the out-of-time test set", x = "False positive rate", y = "True positive
rate") +
  theme_minimal(base_size = 13) + theme(plot.title = element_text(face = "bold"), legend.position =
"right")
ggsave("figs/res_roc_curves.png", p_roc, width = 10, height = 6, dpi = 200)

# Confusion matrices for six representative models
cms <- list("Probit" = pred_probit, "SVM (Linear)" = pred_svm_lin, "Logit" = pred_logit,
            "Random Forest" = pred_rf, "XGBoost" = pred_xgb, "CART" = pred_dt)
cm_df <- do.call(rbind, lapply(names(cms), function(n) {
  t <- as.data.frame(table(Predicted = cms[[n]], Actual = y)); t$Model <- n; t }))
cm_df$Model <- factor(cm_df$Model, levels = names(cms))
p_cm <- ggplot(cm_df, aes(Actual, Predicted, fill = Freq)) + geom_tile(color = "white") +
  geom_text(aes(label = Freq), size = 5, fontface = "bold") +
  scale_fill_gradient(low = "#eaf4f1", high = "#2a9d8f", guide = "none") +
  facet_wrap(~Model, ncol = 3) +
  labs(title = "Confusion matrices (94 test signals)", x = "Actual class", y = "Predicted class") +
  theme_minimal(base_size = 12) + theme(plot.title = element_text(face = "bold"),
                                        axis.text.x = element_text(angle = 20, hjust = 1))
ggsave("figs/res_confusion_matrices.png", p_cm, width = 10, height = 6.5, dpi = 200)

# XGBoost feature importance (gain share)
imp <- xgb_imp; imp$Overall <- 100 * imp$Gain / max(imp$Gain)
p_imp <- ggplot(imp, aes(reorder(Feature, Overall), Overall)) + geom_col(fill = "#2a9d8f", width = 0.65) +
  coord_flip() + labs(title = "XGBoost feature importance (gain)", x = NULL, y = "Relative importance (0-
100)") +
  theme_minimal(base_size = 13) + theme(plot.title = element_text(face = "bold"))
ggsave("figs/res_xgb_importance.png", p_imp, width = 8, height = 4.5, dpi = 200)

# Final 15-model leaderboard
lb15 <- rbind(leaderboard, record_model(15, "Boosting", "XGBoost (5-fold CV grid search)", pred_xgb, y))
%>%
  arrange(desc(Accuracy), desc(Kappa))
```

```
write.csv(lb15, "leaderboard_15_models.csv", row.names = FALSE)
print(lb15[, c("Model", "Accuracy", "Kappa", "Precision", "Sensitivity", "Specificity")], row.names =
FALSE)
cat("\nXGBoost best CV tune:\n"); print(best)

# 15-model accuracy chart
lb15$Model <- factor(lb15$Model, levels = rev(lb15$Model))
p_acc <- ggplot(lb15, aes(Model, Accuracy, fill = Category)) + geom_col(width = 0.72) + coord_flip() +
  geom_text(aes(label = sprintf("%.2f%%", Accuracy)), hjust = -0.1, size = 3.6, fontface = "bold") +
  ylim(0, 110) + scale_fill_brewer(palette = "Set2") +
  labs(title = "Out-of-time accuracy across 15 models", x = NULL, y = "Accuracy (%)", fill = "Model
family") +
  theme_minimal(base_size = 12) + theme(plot.title = element_text(face = "bold"), legend.position =
"bottom")
ggsave("figs/res_accuracy_15_models.png", p_acc, width = 10, height = 7, dpi = 200)
```

11.4 eda_and_statistics.R (EDA figures and statistical tests)

```
# EDA AND STATISTICAL ANALYSIS - Study Stock options project
# Course : Data Analytics Solutions using R, VIT Chennai
# Input  : data/options_trade_signals_sample.csv  (modelling dataset, 500 rows)
#          data/preprocessed_features_sample.csv  (option chain features)
#          data/raw_sample_dataset.csv            (raw WebSocket ticks)
# Output : figs/*.png and stats_output.txt
suppressPackageStartupMessages({
  library(dplyr); library(readr); library(tidyr); library(ggplot2); library(scales)
})

dir.create("figs", showWarnings = FALSE)
theme_rep <- theme_minimal(base_size = 13) +
  theme(plot.title = element_text(face = "bold"), legend.position = "bottom",
        panel.grid.minor = element_blank())
col_ce <- "#1f77b4"; col_pe <- "#d62728"; col_p <- "#2a9d8f"; col_u <- "#e76f51"

# ---- 1. Load and build the economic target (same rule as the benchmark script) ----
df_raw <- read_csv("data/options_trade_signals_sample.csv", show_col_types = FALSE) %>% arrange(timestamp)
friction_drag <- 0.35 * df_raw$spread + 0.15
gross_alpha   <- 0.08 * df_raw$z_vwoim + 0.06 * df_raw$z_oi_flow + 0.20 * (df_raw$confidence - 0.90)
df <- df_raw %>%
  mutate(net_return = gross_alpha - friction_drag,
         target = factor(ifelse(net_return > 0, "Profitable", "Unprofitable"),
                         levels = c("Profitable", "Unprofitable")),
         log_volume = log1p(volume))
feat <- c("confidence", "spread", "log_volume", "z_vwoim", "z_oi_flow")

sink("stats_output.txt", split = TRUE)
cat("=== DATASET OVERVIEW ===\n")
cat("Rows:", nrow(df), " Columns:", ncol(df_raw), "\n")
cat("Underlyings:\n"); print(table(df$underlying))
cat("Option type:\n"); print(table(df$option_type))
cat("Expiries:\n"); print(table(df$expiry))
cat("Missing values per column (modelling dataset):\n"); print(colSums(is.na(df_raw)))
cat("\nClass distribution of target:\n"); print(table(df$target));
print(round(prop.table(table(df$target)), 4))

# ---- 2. Descriptive statistics ----
cat("\n=== DESCRIPTIVE STATISTICS (feature columns) ===\n")
desc <- df %>% select(all_of(feat)) %>%
  pivot_longer(everything(), names_to = "feature") %>%
  group_by(feature) %>%
  summarise(mean = mean(value), sd = sd(value), median = median(value),
            min = min(value), max = max(value), skew = mean((value - mean(value))^3) / sd(value)^3,
            .groups = "drop")
print(as.data.frame(desc), digits = 4)

# ---- 3. Hypothesis tests: features by target class ----
cat("\n=== WELCH T-TEST AND WILCOXON TEST: feature vs target class ===\n")
tests <- lapply(feat, function(f) {
  x <- df[[f]][df$target == "Profitable"]; y <- df[[f]][df$target == "Unprofitable"]
  tt <- t.test(x, y); wt <- suppressWarnings(wilcox.test(x, y))
  data.frame(feature = f, mean_profitable = mean(x), mean_unprofitable = mean(y),
             t = unname(tt$statistic), t_p = tt$p.value, wilcoxon_p = wt$p.value)
})
tests <- do.call(rbind, tests); print(tests, digits = 4, row.names = FALSE)
```

```
# ---- 4. Association tests ----
cat("\n=== CHI-SQUARE: option_type vs target ===\n")
print(chisq.test(table(df$option_type, df$target)))
cat("\n=== CHI-SQUARE: underlying vs target ===\n")
print(chisq.test(table(df$underlying, df$target)))
cat("\n=== ONE-WAY ANOVA: spread by underlying ===\n")
print(summary(aov(spread ~ underlying, data = df)))
cat("\n=== SHAPIRO-WILK NORMALITY (feature columns) ===\n")
for (f in feat) { s <- shapiro.test(df[[f]]); cat(sprintf("%-12s W = %.4f  p = %.3g\n", f, s$statistic,
s$p.value)) }

# ---- 5. Correlation ----
cat("\n=== PEARSON CORRELATION MATRIX ===\n")
cm <- cor(df %>% select(all_of(feat), net_return))
print(round(cm, 3))
cat("\nSelected correlation tests:\n")
for (pr in list(c("spread", "net_return"), c("z_vwoim", "net_return"), c("z_oi_flow", "net_return"),
c("confidence", "net_return"))) {
  ct <- cor.test(df[[pr[1]]], df[[pr[2]]])
  cat(sprintf("%-10s vs %-10s r = %6.3f  p = %.3g\n", pr[1], pr[2], ct$estimate, ct$p.value))
}
sink()

# ---- 6. Figures ----
# Fig 1: class balance
p1 <- ggplot(as.data.frame(table(df$target)), aes(Var1, Freq, fill = Var1)) +
  geom_col(width = 0.6) + geom_text(aes(label = Freq), vjust = -0.4, fontface = "bold") +
  scale_fill_manual(values = c(Profitable = col_p, Unprofitable = col_u), guide = "none") +
  labs(title = "Target class balance (net of friction)", x = NULL, y = "Signals") + theme_rep
ggsave("figs/eda_1_class_balance.png", p1, width = 7, height = 4.5, dpi = 200)

# Fig 2: feature histograms
long <- df %>% select(target, all_of(feat)) %>% pivot_longer(-target, names_to = "feature")
p2 <- ggplot(long, aes(value, fill = target)) +
  geom_histogram(bins = 30, alpha = 0.75, position = "identity") +
  facet_wrap(~feature, scales = "free", ncol = 3) +
  scale_fill_manual(values = c(Profitable = col_p, Unprofitable = col_u), name = NULL) +
  labs(title = "Feature distributions by target class", x = NULL, y = "Count") + theme_rep
ggsave("figs/eda_2_feature_histograms.png", p2, width = 11, height = 6.5, dpi = 200)

# Fig 3: boxplots
p3 <- ggplot(long, aes(target, value, fill = target)) + geom_boxplot(outlier.size = 1, alpha = 0.8) +
  facet_wrap(~feature, scales = "free_y", ncol = 5) +
  scale_fill_manual(values = c(Profitable = col_p, Unprofitable = col_u), guide = "none") +
  labs(title = "Feature spread by target class", x = NULL, y = NULL) + theme_rep +
  theme(axis.text.x = element_text(angle = 25, hjust = 1))
ggsave("figs/eda_3_boxplots.png", p3, width = 12, height = 4.8, dpi = 200)

# Fig 4: correlation heatmap
cl <- as.data.frame(as.table(cm)); names(cl) <- c("a", "b", "r")
p4 <- ggplot(cl, aes(a, b, fill = r)) + geom_tile(color = "white") +
  geom_text(aes(label = sprintf("%.2f", r)), size = 4) +
  scale_fill_gradient2(low = col_pe, mid = "white", high = col_ce, limits = c(-1, 1), name = "r") +
  labs(title = "Correlation matrix: features and net return", x = NULL, y = NULL) + theme_rep +
  theme(axis.text.x = element_text(angle = 30, hjust = 1))
ggsave("figs/eda_4_correlation.png", p4, width = 8, height = 6, dpi = 200)

# Fig 5 and 6: option chain features (recorded NIFTY / BANKNIFTY)
pf <- read_csv("data/preprocessed_features_sample.csv", show_col_types = FALSE)
nifty <- pf %>% filter(underlying == "NIFTY") %>% group_by(strike, option_type) %>%
  summarise(oi = max(oi, na.rm = TRUE), .groups = "drop")
p5 <- ggplot(nifty, aes(factor(strike), oi, fill = option_type)) +
  geom_col(position = "dodge", width = 0.75) +
  scale_fill_manual(values = c(CE = col_ce, PE = col_pe), name = "Option type") +
  scale_y_continuous(labels = label_number(scale_cut = cut_short_scale())) +
  labs(title = "NIFTY open interest by strike (recorded session, 18 Jun 2026)", x = "Strike", y = "Open
interest") +
  theme_rep + theme(axis.text.x = element_text(angle = 45, hjust = 1))
ggsave("figs/eda_5_open_interest.png", p5, width = 10, height = 5, dpi = 200)

pm <- pf %>% filter(spot > 0, bid > 0, ask > 0, underlying == "NIFTY") %>%
  mutate(moneyness = strike / spot, spread = ask - bid)
p6 <- ggplot(pm, aes(moneyness, spread, color = option_type)) + geom_point(alpha = 0.55, size = 1.8) +
  geom_vline(xintercept = 1, linetype = "dashed") +
  scale_color_manual(values = c(CE = col_ce, PE = col_pe), name = "Option type") +
  labs(title = "Bid-ask spread vs moneyness (NIFTY, recorded ticks)", x = "Strike / spot", y = "Spread
```

```
(INR)") + theme_rep
ggsave("figs/eda_6_spread_moneyness.png", p6, width = 10, height = 5, dpi = 200)

# Fig 7: feed latency
rw <- read_csv("data/raw_sample_dataset.csv", show_col_types = FALSE)
lat <- rw %>% filter(!is.na(latency_ms), abs(latency_ms) < 5000)
p7 <- ggplot(lat, aes(latency_ms)) + geom_histogram(bins = 40, fill = col_ce, color = "white") +
  geom_vline(xintercept = median(lat$latency_ms), color = col_u, linewidth = 1) +
  labs(title = "WebSocket feed latency (receive minus exchange time)",
       subtitle = sprintf("Median %.0f ms across %d raw ticks", median(lat$latency_ms), nrow(lat)),
       x = "Latency (ms)", y = "Ticks") + theme_rep
ggsave("figs/eda_7_latency.png", p7, width = 8, height = 4.8, dpi = 200)

cat("Raw ticks:", nrow(rw), " Preprocessed rows:", nrow(pf), " Signals:", nrow(df_raw), "\n")
message("EDA figures and stats_output.txt written")
```

12. GitHub Repository

https://github.com/harsh01103/stockezee

The repository holds the Python market-data engine, the Node gateway, the React frontend (Study Stock), the R scripts, the sample data, documentation with screenshots, and a GitHub Actions workflow.